



Software Quality Analytics And Risk Assessment Using Cloud-Based Machine Learning

A PROJECT REPORT

Submitted by

PRASANNARAM R R

[723722104112]

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

V.S.B COLLEGE OF ENGINEERING TECHNICAL CAMPUS

(An Autonomous Institution)

COIMBATORE - 642109, TAMIL NADU.

ANNA UNIVERSITY:

CHENNAI 600 025

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project report “**Software Quality Analytics And Risk Assessment Using Cloud-Based Machine Learning**” is the Bonafide work of the following student, **Prasannaram R R [723722104112]** who carried out the project work under my supervision.

SIGNATURE

Mrs. V. RADHA, M.E.,

HEAD OF THE DEPARTMENT,

DEPARTMENT OF COMPUTER

SCIENCE AND ENGINEERING,

V.S.B. COLLEGE OF ENGINEERING

TECHNICAL CAMPUS,

Coimbatore - 642109

SIGNATURE

Dr. R. JAMUNA, M.E., Ph.D.,

SUPERVISOR,

PROFESSOR

DEPARTMENT OF COMPUTER

SCIENCE AND ENGINEERING,

V.S.B. COLLEGE OF ENGINEERING

TECHNICAL CAMPUS,

Coimbatore – 642109

Submitted for the university examination held on

.....

V.S.B. College of Engineering Technical Campus, Coimbatore.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

First of all, we extend our heartfelt gratitude to the management of V.S.B. College of Engineering Technical Campus institutions, Chairman **Mr. V. S. BALSAMY B.Sc., L.L.B.**, for providing us with all sorts of support in the completion of this project.

We record our indebtedness to our beloved Principal **Dr. V. VELMURUGAN, M.E., Ph.D.**, for his guidance and sustained encouragement for the successful completion of the project.

We express our sincere gratitude to our respected Vice Principal **Dr. P. RAJU, M.E., Ph.D.**, for his valuable guidance, constant encouragement, and unwavering support throughout the development of this project. His insightful suggestions and motivation greatly contributed to the successful completion of our work.

We express our sincere gratitude to our respect Dean Academic **Dr. M. VARATHARAJ, M.E., Ph.D.**, for his valuable guidance, constant encouragement, and insightful suggestions throughout the course of this project. His continuous support and motivation greatly contributed to the successful completion of our work.

We are highly grateful to **Dr. T. KALAIKUMURAN M.E., Ph.D.**, Head of the Department, Department of Computer Science and Engineering, for his valuable suggestions throughout the course of the project.

We also extend our sincere thanks to **Mrs. V. RADHA, M.E.**, Head of the Department, for her continued guidance and support.

We also extend our sincere thanks to **Dr. R. JAMUNA, M.E., Ph.D.**, Professor and project guide, for her kind support and timely assistance rendered which led us to the successful completion of our project. Her technical expertise, encouragement, and insightful feedback at every stage were invaluable.

We extend our heartfelt salutation to our beloved parents and friends who have always been an integral part in helping us through the tough times, all faculty members and non- teaching staffs for providing their moral support making herculean success of our project.



V.S.B. College of Engineering Technical Campus (An Autonomous Institution)

Approved by AICTE, New Delhi, Affiliated to Anna University, Chennai
NAAC Accredited Institution, NBA Accredited Courses
Coimbatore to Pollachi Road NH -209, Ealur Privu, Kinathukadavu Taluk,
Coimbatore - 642109, Tamilnadu, India. Email:office@vsbcetc.com website : www.vsbcetc.com

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION OF THE INSTITUTION:

We endeavor to impart futuristic technical education of the highest quality to the student community and to inculcate discipline in them to face the world with self-confidence and thus we prepare them for life as responsible citizens to uphold human values and to be of services at large. We strive to bring up the Institution as an Institution of Academic excellence of International standard.

MISSION OF THE INSTITUTION:

We transform persons into personalities by the state-of-the-art infrastructure, time consciousness, quick response and the best academic practices through assessment and advice.


PRINCIPAL



V.S.B. College of Engineering Technical Campus **(An Autonomous Institution)**

Approved by AICTE, New Delhi, Affiliated to Anna University, Chennai
NAAC Accredited Institution, NBA Accredited Courses
Coimbatore to Pollachi Road NH -209, Ealur Privu, Kinathukadavu Taluk,
Coimbatore - 642109, Tamilnadu, India. Email:office@vsbcetc.com website : www.vsbcetc.com

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION OF THE DEPARTMENT:

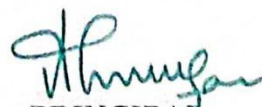
To offer a quality education in computer science and Engineering, encourage life-long learning and make graduates responsible for society by upholding social values in the field of emerging technology.

MISSION OF THE DEPARTMENT:

The Department strives to contribute to the expansion of knowledge in the discipline of Computer Science and Engineering and aims:

- To produce graduates with sound technical knowledge and good skills that prepare them for a rewarding career in prominent industries.
- To promote collaborative learning and research with Industry, Government and International organizations for continuous knowledge transfer and enhancement.
- To promote entrepreneurship and mould the graduates to be leaders by cultivating the spirit of social and ethical values.


HOD


PRINCIPAL



V.S.B. College of Engineering Technical Campus (An Autonomous Institution)

Approved by AICTE, New Delhi, Affiliated to Anna University, Chennai
NAAC Accredited Institution, NBA Accredited Courses
Coimbatore to Pollachi Road NH -209, Ealur Privu, Kinathukadavu Taluk,
Coimbatore - 642109, Tamilnadu, India. Email: office@vsbcetec.com website : www.vsbcetec.com

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROGRAM OUTCOMES

PO1

Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2

Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3

Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4

Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5

Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6

The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7

Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8

Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9

Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10

Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11

Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12

Life Long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.


HOD


PRINCIPAL



V.S.B. College of Engineering Technical Campus

(An Autonomous Institution)

Approved by AICTE, New Delhi, Affiliated to Anna University, Chennai

NAAC Accredited Institution, NBA Accredited Courses

Coimbatore to Pollachi Road NH -209, Ealur Privu, Kinathukadavu Taluk,
Coimbatore - 642109, Tamilnadu, India. Email: office@vsbcetc.com website : www.vsbcetc.com

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1 - Graduates will have successful careers with high level of technical competency and problem-solving skills to produce innovative solutions for industrial needs.

PEO2 – Graduates will have good professionalism, team work, effective communication, leadership qualities and life-long learning for the welfare of mankind.

PEO3 – Graduates will be familiar with recent trends in industry for delivering and implementing innovative system in collaboration.

PROGRAM SPECIFIC OUTCOME (PSO)

PSO1

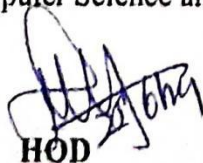
Addressing societal problems through design and development of software and firmware solutions using latest Computer Science tools and technologies.

PSO2

Involving students enthusiastically in software development, software testing, storage, computing and business intelligence sectors.

PSO3

Making them to use their technical expertise in latest technologies and update knowledge continuously in Computer Science and Engineering to excel in career.


HOD


PRINCIPAL

ABSTRACT

This title “Software Quality Analytics and Risk Assessment Using Cloud-Based Machine Learning” is develop with a modern web-based architecture. The frontend technologies used in this project include HTML, CSS, Bootstrap, and JavaScript for creating an interactive user interface. The backend is implemented using Python with the Flask framework to manage server-side operations and system functionality. This project focuses on Software Quality Analytics and Risk Level Prediction using machine learning techniques. The main objective is to analyze important software metrics and predict whether a software module falls under Low, Medium, or High risk. In real-world software development, identifying risk at an early stage is very important to reduce defects and project failure. The system uses key input metrics such as Coupling Between Objects (CBO), Fan-in, Fan-out, Weighted Methods per Class (WMC), Depth of Inheritance Tree (DIT), Number of Children (NOC), and Response for Class (RFC). It also considers cohesion and size-related metrics like Lack of Cohesion of Methods (LCOM) and Lines of Code (LOC). In addition, complexity indicators such as Loop Quantity, Comparisons Quantity, and Maximum Nested Blocks are included. These features provide a clear understanding of code complexity, maintainability, and dependency levels. The collected metrics are used to train a machine learning model using the Random Forest algorithm. The model learns patterns from the data and predicts the risk level of new software inputs. The system classifies outputs into three categories: Low Risk, Medium Risk, and High Risk. A web-based interface is developed to allow users to input these metrics easily and obtain instant predictions. The system also displays graphical results to visualize prediction probabilities. This helps developers understand how different factors contribute to software risk. By predicting risks early, the system helps improve software quality and reduces debugging time.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	ABSTRACT	i
	LIST OF DIAGRAMS	ii
	LIST OF FIGURES	iii
	LIST OF SYMBOLS AND ABBREVIATIONS	iv
1	INTRODUCTION	1-3
2	LITERATURE REVIEW	4-10
3	SYSTEM ANALYSIS	11-15
	3.1 PROBLEM STATEMENT	
	3.2 EXISTING METHODOLOGY	
	3.3 DISADVANTAGES OF EXISTING WORK	
	3.4 PROPOSED METHODOLOGY	
	3.4.1 ADVANTAGES	
	3.5 SYSTEM ARCHITECTURE	
	3.6 WORKING OF THE SYSTEM	
	3.7 LIMITATION OF THE SYSTEM	
4	SYSTEM SPECIFICATIONS	16
	4.1 HARDWARE SPECIFICATION	
	4.2 SOFTWARE SPECIFICATION	
5	SYSTEM FEATURES	17-29

5.1 INTRODUCTION TO OPERATING SYSTEM

5.2 INTRODUCTION TO BACK END

5.3 MODULE DESCRIPTION

5.3.1 DATA COLLECTION MODULE

5.3.2 DATA PREPROCESSING MODULE

5.3.3 MODULE DESCRIPTION

5.3.4 MACHINE LEARNING MODEL MODULE

5.3.5 PREDICTION MODULE

5.3.6 WEB INTERFACE MODULE

6 METHODOLOGY 30-38

6.1 DATA COLLECTION

6.2 DATA PREPROCESSING

6.3 FEATURE SELECTION

6.4 RISK LABEL CREATION

6.5 MODEL TRAINING

6.6 MODEL TESTING AND EVALUATION

6.7 PREDICTION

6.8 VISUALIZATION

6.9 WEB INTERFACE

6.10 SYSTEM DIAGRAMS

6.10.1 DATA FLOW DIAGRAM (DFD)

6.10.2 DFD LEVEL 1

6.10.3 SYSTEM ARCHITECTURE DIAGRAM

6.10.4 METHODOLOGY DIAGRAM

6.10.5 USE CASE DIAGRAM

	6.11 RANDOM FOREST ALGORITHM	
7	SYSTEM TESTING AND IMPLEMENTATION	39-45
	7.1 SYSTEM TESTING	
	7.1.1 UNIT TESTING	
	7.1.2 INTEGRATION TESTING	
	7.1.3 SYSTEM TESTING	
	7.1.4 FUNCTIONAL TESTING	
	7.1.5 PERFORMANCE TESTING	
	7.1.6 USER INTERFACE TESTING	
	7.1.7 VALIDATION TESTING	
	7.1.8 ERROR TESTING	
	7.2 SYSTEM IMPLEMENTATION	
8	SYSTEM MAINTANANCE	46-49
	8.1 OVERVIEW	
	8.2 TYPES OF MAINTANANCE	
	8.3 MAINTANANCE WORKFLOW	
9	SYSTEM DESIGN	50-53
	9.1 INPUT DESIGN	
	9.2 OUTPUT DESIGN	
	CONCLUSION	54-56
	FUTURE ENHANCEMENT	57-60
	REFERENCES	61
	APPENDIX - I	62-69
	APPENDIX - II	70-80

LIST OF DIAGRAMS

FIGURE NO	TITLE	PAGE NO
6.10.1	DATA FLOW DIAGRAM (DFD) LEVEL 0	34
6.10.2	DFD LEVEL 1	34
6.10.3	SYSTEM ARCHITECTURE DIAGRAM	35
6.10.4	METHODOLOGY DIAGRAM	36
6.10.5	USECASE DIAGRAM	37
8.3	WORKFLOW DIAGRAM	49

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
9.1	INPUT AND OUTPUT DESIGN	53
10.1	FUTURE ENHANCEMENT	60
12.1.1	ACCURACY LEVEL	62
12.1.2	CONFUSION MATRIX	63
12.1.3	FEATURE IMPORTANCES	64
12.2.1	HOME PAGE	65
12.2.2	ABOUT PAGE	66
12.2.3	PREDICT PAGE	67
12.2.4	VALUE ENTER PAGE	68
12.2.5	RESULTS PAGE	69

LIST OF SYMBOLS AND ABBREVIATIONS

S. NO.	ABBREVIATION	FULL FORM
1.	ML	Machine Learning
2.	RF	Random Forest
3.	CBO	Coupling Between Objects
4.	WMC	Weighted Methods per Class
5.	DIT	Depth of Inheritance Tree
6.	NOC	Number of Children
7.	RFC	Response for Class
8.	LCOM	Lack of Cohesion of Methods
9.	LOC	Lines of Code
10.	HTML	HyperText Markup Language
11.	CSS	Cascading Style Sheets
12.	JS	JavaScript
13.	UI	User Interface
14.	API	Application Programming Interface
15.	IDE	Integrated Development Environment
16.	CMM	Capability Maturity Model

CHAPTER – 1

INTRODUCTION

Software quality plays a crucial role in the success of modern software systems. As software applications become more complex, maintaining high quality and reliability has become a major challenge for developers and organizations. Poor software quality can lead to defects, system failures, increased maintenance costs, and user dissatisfaction. Therefore, it is essential to identify potential risks in software systems at an early stage of development to ensure better performance and reliability. Traditional methods of software testing and quality assurance often detect issues only after development or deployment. This leads to increased time consumption and higher costs for fixing errors. To overcome these limitations, advanced techniques such as machine learning can be used to predict software risks in advance. Machine learning models can analyze historical data and identify patterns that indicate potential defects or failures, making the development process more efficient.

In this project, a machine learning-based approach is proposed to perform software quality analytics and risk assessment. The system uses important software metrics such as coupling, cohesion, complexity, and code size to evaluate the quality of software modules. A Random Forest algorithm is used to train the model and classify the software into Low, Medium, and High risk levels. This helps developers understand which parts of the software require more attention and improvement. Furthermore, the project includes a user-friendly web interface developed using Flask, where users can input software metrics and receive instant predictions. The system also provides graphical visualization of the prediction results, making it easier to interpret. This approach not only improves software quality but also reduces development cost and time by enabling early risk detection. Overall, the project demonstrates the effective use of machine learning techniques in software engineering for better

decision-making and risk management. The primary objective of this project is to develop a system that can analyze software metrics and predict the risk level of software modules using machine learning techniques. The system aims to classify software into Low, Medium, and High risk categories based on important features such as coupling, cohesion, complexity, and code size. By using the Random Forest algorithm, the project seeks to build an accurate and reliable prediction model that can assist developers in identifying potential defects at an early stage. This helps in improving software quality and reducing the chances of failure.

Another objective of the project is to create a user-friendly web interface that allows users to easily input software metrics and obtain prediction results instantly. The system also aims to provide graphical visualization of prediction probabilities to enhance understanding and decision-making. By integrating machine learning with a web-based platform, the project focuses on delivering a practical and scalable solution for real-time software risk assessment. Overall, the objective is to support developers and project managers in making better decisions, minimizing development cost, and ensuring high-quality software systems.

The scope of this project focuses on analyzing software quality using machine learning techniques to predict risk levels in software modules. It is designed to work with software metrics such as coupling, cohesion, complexity, and code size to identify potential issues in the early stages of development. The system can be applied to various types of software projects where quality assessment is important. It helps developers understand which parts of the code are more prone to defects and require improvement. This project mainly covers risk prediction using historical data and does not depend on real-time data integration.

In addition, the project includes a web-based application that allows users to input software metrics and receive instant predictions along with graphical visualization. The system is scalable and can be extended to include more advanced machine learning algorithms and larger datasets. It can also be integrated with cloud platforms for real-time analytics in the future. However, the current scope is limited to predefined input features and a trained model using the Random Forest algorithm. Overall, the project provides a foundation for improving software quality and can be enhanced further for industrial-level applications. This project presents a system for Software Quality Analytics and Risk Prediction using machine learning techniques. The main idea is to analyze software metrics and determine the risk level associated with software modules. Various input features such as coupling, cohesion, complexity, and size-related metrics are used to evaluate the quality of the code. These metrics help in understanding how complex and maintainable the software is. Based on these inputs, the system predicts whether the software falls under Low, Medium, or High risk categories. The project uses the Random Forest algorithm to build an effective prediction model. The system is implemented with a web-based interface using Flask, allowing users to input values and obtain instant results. It also provides graphical visualization of prediction probabilities for better understanding. The overall workflow includes data collection, preprocessing, model training, and prediction. This approach helps in identifying potential risks early, improving software quality, and supporting better decision-making during development. Furthermore, the system is designed to be scalable and adaptable for different types of software projects. It can handle multiple input features efficiently and provide consistent predictions. The modular architecture allows easy integration of new algorithms and additional metrics in the future. By combining machine learning with a web-based interface, the system provides a practical and user-friendly solution for software risk assessment.

CHAPTER – 2

LITERATURE REVIEW

[1] "Software Quality Cost Research: A Systematic Review," IEEE Access.

For decades, users of software solutions have been suffering from poor solution quality (Whittaker and Voas, 2002). Over the years, quality has emerged to be a key issue in software development (Pralhad and Krishnan, 1999). Software vendors have attempted to tackle this issue by adapting concepts from other engineering disciplines, such as manufacturing (Antony and Fergusson, 2004). There, approaches ranging from Total Quality Management over Six Sigma and Kaizen to Lean Production have led to significant gains in productivity and quality. To attain similar results in software development, many of these concepts have been adapted and tailored to its characteristics (Middleton and Sutton, 2005). In this quest for higher productivity and quality, the economics of software engineering are of particular interest (Boehm, 1981). While some economic aspects, such as software development effort estimation (Jorgensen and Shepperd, 2007) and software process improvement (Hansen et al., 2004), have frequently been discussed, others have received less attention. Indeed, little research has specifically been devoted to those costs which are “incurred in the pursuit of [software] quality or in performing quality-related activities” (Pressman, 2010, p. 407). This is remarkable, because software vendors typically spend 30– 50% of their development budget on defect detection and correction (Ebert and Dumke, 2010).

[2] "Systematic Literature Review on Software Quality Cost Analysis," IEEE Transactions on Software Engineering.

In most engineering disciplines, literature studies summarizing the latest research results on quality costs are regularly published (e.g., Williams et al., 1999, Schiffauerova and Thomson, 2006). To the best of our knowledge, this is not the case in software engineering; no review prior to ours has in particular been devoted to software quality cost research. However, several studies published in the broader field of software quality and software economics have also covered some quality cost aspects. For instance, the survey of software quality assurance research by Rai et al. (1998) considers software quality costs among other economic aspects, and Jorgensen and Shepperd (2007) systematically review work on software development effort estimation including approaches applicable to software quality cost estimation.

[3]"Software Quality Cost Research: Structured Review and Future Directions," IEEE Access.

This article tries to close this research gap. It is exclusively devoted to software quality cost research. Our objective is to systematically review and structure the existing body of research on software quality costs and to identify areas for future research. We analyze 87 articles published between 1980 and 2009 in 60 leading computing journals for answering eight research questions. These research questions are directed at the research domain in general as well as at specifics of the existing research, regarding the software quality cost categories examined, the scope of investigation, and the research approaches employed.

**[4]"Software Quality Assurance and Testing in Contemporary Development,"
IEEE Access.**

This thesis explores the intricacies of SQA and testing in contemporary software development. It analyzes current approaches, emphasizing their advantages and drawbacks, and discusses how conventional testing techniques may find it challenging to keep pace with fast technological progress. An essential factor involves comprehending the role of SQA in upholding functional and technical standards, as outlined by Galin and Smith, and examining internal and external quality aspects (Galin, 2004).

The project also tackles obstacles like strict timeframes and scarce resources, as pointed out by (Smith, Johnson, & Williams, 2021) ,which may impede thorough testing. The objective is to conduct a thorough evaluation of present and past QA and testing methods in the software industry. This study seeks to identify challenges and limitations in software quality methodologies over time and provide insights into current approaches. This effort is crucial for gaining a more thorough comprehension of the complexities required to guarantee software reliability, scalability, and meeting user standards in the current highly competitive environment.

Furthermore, this thesis highlights the growing importance of intelligent and data-driven quality assurance techniques in modern software engineering. With the emergence of cloud computing, machine learning, and automated analytics, organizations are increasingly adopting advanced tools to improve defect prediction, risk assessment, and testing efficiency. The study examines how cloud-based learning models can support continuous monitoring, faster bug detection, and improved decision-making during the software development lifecycle.

**[5] "Historical Evolution of Software Engineering and Quality Significance,"
IEEE Software.**

Kan (2003) discussed the historical background of software engineering's evolution and growing significance of software quality over time. In the 1960's and 70's before, it could be seen as the period when Information technology (IT) catered to institutional requirements by connecting software to the daily functions of Institutions. The 1970s can be seen as when industries faced significant delays and increased cost in their projects, with an emphasis on planning and controlling of software projects. The 1980s can be seen as the cost era where costs continued to decline, and IT was widely spread among different organizations and Institutions and soon became available to Individual users. The 1990s can be viewed as a time of increased quality and productivity (Kan, 2003).

**[6] "Evolution of Quality Assurance Practices in Software Development,"
IEEE Access.**

Earlier when software development was just introduced, the act of QA was not as meaningful as today. Back then QA was just a formality for making ensure that software meets the development purposes. Companies were doing QA simultaneously with software development, but it was just a post checking without the capacity of making any changes in case if an error occurred (Tummala, 2022).

**[7] "Software Quality Assurance as a Critical Factor for Corporate Success,"
IEEE Access**

Based on historical evidence, we can determine that the present time is considered a quality stage era, where SQA and testing have become dominant, and 'quality' is now the most crucial factor for corporate success and growth. Software companies are finding it increasingly difficult to create bug free software, and without the right tools, projects are at risk of failing. Even a minor flaw could result in significant software failures, resulting in substantial financial losses for businesses and detrimental effects on the quality of the software product. To achieve quality software, testing has become the prominent technique across the industry. Therefore, it is essential for software development to be more effective, ensuring that the quality of the delivered software meet customer expectations to achieve success (Chopde, 2011).

**[8] "Software Quality Definitions, Standards, and Development Compliance,"
IEEE Access.**

Burnstein (2006), cited the IEEE standard glossary of software engineering terminology which defines software quality as the level at which a system, component, or process meets specified requirements and satisfies needs and expectations of customer or user. Compliance with clearly defined performance requirements and functional, documented development standards, and implicit characteristics is expected in all professionally developed software (Pressman, 2005). Further elaborated on the importance of incorporating quality into software development, consistently assessing it, and making necessary corrections as needed (Godbole, 2004). 9 In software development, ensuring the quality is based on the various components of quality. Numerous software

quality factors models and their classification have been proposed over time. It is crucial to assess software quality projects consistently with efficient quality assurance processes. The methods aim to stop defects and testing systems and processes for removing defects. This will allow the development of high-quality software with the desired features and successful delivery.

**[9] "Capability Maturity Model and ISO Standards in Software Process Quality,"
IEEE Access.**

The Capability Maturity Model (CMM) is a technique employed to enhance a company's software development approach. It describes five levels of processes that mature in a structured and systematic way as they progress. CMM outlines five levels: initial, repeatable, defined, managed, and optimizing. This model is like ISO 9001, part of the ISO 9000 standards. ISO 9001 has a basic standard for software process quality, whereas the CMM creates a framework for ongoing enhancement and is more detailed than ISO in outlining the approaches to reach this objective (Al-Qutaish & Rafa, 2010).

**[10] "Software Testing Standards, Defect Detection, and Functional Validation,"
IEEE Access.**

Testing is described in the ANSI/IEEE 1059 standard as examining a software item to detect differences between present and desired states (defects, errors, bugs) and evaluating the functionalities of the software product (Singh & Singh, 2012). Software testing is the primary stage of software development. It is utilized for assessing the caliber, performance, and dependability of the software product. Conducting functional tests on the software helps uncover flaws or inconsistencies that may exist between user expectations and the actual results. It is crucial for this approach that the software meets the designated

requirements and functions correctly to avoid potential risks of failure and vulnerability.

[11] "Impact of Thorough Software Testing on User Experience and Cost Reduction," IEEE Access.

Testing that is thorough leads to a user experience improvement through delivering a product that has a stable performance and is both time and energy efficient. With the help of testing, the overall cost of developing software and product release time can be significantly reduced by early identification of errors. Be it conducted manually or the automated way, software testing is vital for validating functionality, improving quality, and safeguarding reputation so that users can have high confidence in the software performance resulting in customer satisfaction and business success

[12] "Modular System Architecture for Scalable and Maintainable Software Design," IEEE Access.

Furthermore, the modular structure of the system ensures flexibility, scalability, and ease of maintenance. Each module operates independently while maintaining proper coordination with other components. This design approach allows future enhancements, such as integrating advanced machine learning models or additional software metrics, without affecting the overall system. The clear separation of modules also simplifies debugging and improves system performance. Overall, the modular architecture contributes to the efficiency, reliability, and adaptability of the proposed system.

CHAPTER – 3

SYSTEM ANALYSIS

3.1 PROBLEM STATEMENT

To overcome these problems, there is a need for a machine learning-based solution that can automatically analyze software metrics and predict risk levels. The system should be capable of processing input features such as coupling, cohesion, complexity, and code size to determine whether a software module is at Low, Medium, or High risk. It should provide quick and accurate predictions that help developers take preventive actions before issues arise. Additionally, the system should include a user-friendly interface where users can easily input data and view results without technical difficulty. Visualization of prediction results is also important to enhance understanding and interpretation. The solution must be scalable so that it can be applied to different types of software projects. It should also support future enhancements such as integration with cloud platforms and real-time data analysis. By implementing such a system, organizations can reduce development costs, improve efficiency,

In modern software development, maintaining high quality and reliability has become a significant challenge due to the increasing complexity of software systems. As applications grow in size and functionality, it becomes difficult for developers to manually track and identify potential defects or risks in the code. Traditional testing and quality assurance methods are often performed at later stages of development, which makes it costly and time-consuming to fix errors. Many times, critical issues such as high coupling, low cohesion, and excessive complexity go unnoticed until the software is deployed. This leads to system failures, performance issues, and poor user experience. Moreover, developers may not have a clear understanding of which modules are more prone to defects. Without proper risk assessment, project managers find it difficult to allocate

resources effectively. Existing approaches also lack automation and depend heavily on human effort, which increases the chances of errors. As a result, there is a need for an intelligent system that can analyze software metrics and predict risks at an early stage. Such a system should be able to handle large amounts of data and provide accurate results. It should also reduce manual effort and improve decision-making. Therefore, addressing these challenges is essential to improve software quality and reliability, and ensure better software quality. Ultimately, the goal is to create a smart and automated risk assessment tool that supports modern software engineering practices.

Furthermore, the increasing complexity of modern software systems demands intelligent and automated solutions for effective risk analysis. Traditional approaches are not sufficient to handle large-scale data and dynamic development environments. Hence, there is a strong need for advanced techniques that can improve accuracy, reduce manual effort, and support better decision-making in software development.

3.2 EXISTING METHODOLOGY

The existing system for software quality assessment mainly relies on traditional testing and manual analysis techniques. Developers use methods such as code reviews, unit testing, and static analysis tools to identify defects in the software. These approaches are often time-consuming and require significant human effort. In many cases, issues are detected only after the development process or during deployment. This leads to increased cost and delay in fixing errors. Traditional systems do not provide early prediction of software risks based on historical data. They lack automation and intelligence in analyzing complex software metrics.

The existing system for software quality assessment mainly relies on traditional testing and manual analysis techniques. Developers use methods such as code reviews, unit testing, and static analysis tools to identify defects in the software. These approaches are often time-consuming and require significant human effort. In many cases, issues are detected only after the development process or during deployment. This leads to increased cost and delay in fixing errors. Traditional systems do not provide early prediction of software risks based on historical data. They lack automation and intelligence in analyzing complex software metrics.

Moreover, these methods do not effectively handle large-scale data generated in modern software projects. The existing systems also fail to identify hidden patterns that indicate potential defects. There is no proper mechanism to classify software modules into different risk levels. Decision-making depends heavily on developer experience rather than data-driven insights. As a result, software quality may be compromised. Hence, the existing system is not efficient in providing accurate and early risk assessment. Overall, the existing methodologies lack efficiency, scalability, and the ability to provide early risk prediction. These limitations highlight the need for a more advanced and automated approach to improve software quality assessment.

3.3 DISADVANTAGES OF EXISTING WORK

- Does not predict software risks at an early stage.
- Highly dependent on manual testing and human effort.
- Time-consuming process for detecting defects.
- Increased development cost due to late error detection.
- Cannot handle large and complex datasets efficiently.
- Lacks automation and intelligent decision-making.
- Does not use historical data for analysis.
- Unable to identify hidden patterns in software metrics.
- No proper classification of risk levels (Low, Medium, High).
- Leads to reduced software quality and reliability.

3.4 PROPOSED METHODOLOGY

The proposed system introduces a machine learning-based approach for software quality analysis and risk prediction. It uses important software metrics such as coupling, cohesion, complexity, and code size to evaluate software modules. The system applies the Random Forest algorithm to train a predictive model using historical data. This model is capable of classifying software into Low, Medium, and High risk levels. Unlike traditional methods, the proposed system provides early risk detection, helping developers take preventive actions. It reduces manual effort by automating the analysis and prediction process. The system also includes a user-friendly web interface where users can input software metrics easily. It provides instant prediction results along with graphical visualization for better understanding. The approach improves software quality, reduces development time, and minimizes costs. Overall, the proposed system offers an efficient and intelligent solution for modern software risk assessment.

3.4.1 ADVANTAGES

- Provides early prediction of software risks.
- Reduces manual effort through automation.
- Improves software quality and reliability.
- Helps in identifying defect-prone modules.
- Saves development time and cost.
- Uses machine learning for accurate predictions.
- Handles large datasets efficiently.
- Provides clear classification (Low, Medium, High risk).
- Offers graphical visualization for better understanding.
- Supports better decision-making for developers and managers.

3.5 SYSTEM ARCHITECTURE

The system architecture represents the overall structure of the proposed system. It illustrates how input software metrics are processed through different stages such as preprocessing, model training, and prediction. The architecture ensures smooth data flow between components and efficient system operation. It provides a clear understanding of how the system functions as a whole. This helps in improving design clarity and system performance.

3.6 WORKING OF THE SYSTEM

The working of the system begins with collecting software metrics from the user or dataset. The input data is preprocessed to remove inconsistencies and ensure quality. Relevant features are selected and passed to the trained machine learning model. The model analyzes the input and predicts the risk level as Low, Medium, or High. The result is then displayed to the user along with graphical visualization. This process ensures accurate and efficient software risk prediction.

3.7 LIMITATION OF THE SYSTEM

Despite its advantages, the system has certain limitations. The prediction accuracy depends on the quality and size of the dataset used for training. The system currently supports a limited set of software metrics. It may not perform well for completely new or unseen types of software projects. Additionally, the model requires proper input values to generate accurate results. These limitations can be addressed in future enhancements.

CHAPTER - 4

SYSTEM SPECIFICATIONS

4.1 HARDWARE SPECIFICATION

Processor	: Intel icore 7 5 th gen
Hard disk	: 500 GB
Ram	: 12 GB
Keyboard	: Logitech of 104 keys
Mouse	: Logitech mouse
Monitor	: 14 inch samtron monitor
GPU	: NVIDIA Geforce GTX 1650

4.2 SOFTWARE SPECIFICATION

FRONT-END	:	Html , Css, Javascript
BACK-END	:	Python
FRAMEWORK	:	Flask
OPERATING SYSTEM	:	Windows 10
TOOLS	:	Python IDLE

CHAPTER - 5

SYSTEM FEATURES

5.1 INTRODUCTION TO OPERATING SYSTEM

Windows 10 is an operating system. An operating system belongs to a special category of software called system software and performed three major functions.

- ✓ It acts like the resource manager of the computer i.e. it controls and coordinates the various resource of the computer like memory (RAM), storage devices (floppy disk and hard disk), input and output devices (keyboard, mouse, monitor, printer etc.)
- ✓ It acts as an interpreter between other software and hardware.
- ✓ It manages all our files and folders and allows us to do various file/disk.

GRAPHICAL USERINTERFACE

Windows 10 is a GUI (graphical user interface) based operating system. As the name indicates, GUI means interfacing through graphical methods with the users. Windows 10 is easier and faster as it loaded on 64 bit microprocessor computer, this supports graphical representations.

FEATURES OF WINDOWS 10

- ✓ Windows 10 is considered more user friendly than its counterpart MS DOS. It can support long file name as compared to the eight

letter filename supported by DOS. Starting in Windows 10, files have name up to 255 characters long.

- ✓ A web browser called with internet explorer comes as part of Windows 10. It offers us to access to a vast collection of world knowledge (web is collection of documents) through a worldwide conglomerated if numerous computer networks.
- ✓ Windows 10 provides internet collaboration through it is yet another feature called net meeting. It helps us in working together, sharing information, and exchanging files and documents during conference using data, audio and video.

5.2 INTRODUCTION TO BACK END

PYTHON

Python is an interpreter, object-oriented, high-level programming language with dynamic semantics. Its high-level built in data structures, combined with dynamic typing and dynamic binding; make it very attractive for Rapid Application Development, as well as for use as a scripting or glue language to connect existing components together. Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance. Python supports modules and packages, which encourages program modularity and code reuse. The Python interpreter and the extensive standard library are available in source or binary form without charge for all major platforms, and can be freely distributed.

Python is Free

The Python interpreter is developed under an OSI-approved open-source license, making it free to install, use, and distribute, even for commercial purposes. A version of the interpreter is available for virtually any platform there is, including all flavors of Unix, Windows, MAC OS, smart phones and tablets, and probably anything else you ever heard of. A version even exists for the half dozen people remaining who use OS/2.

Python is Portable

Because Python code is interpreted and not compiled into native machine instructions, code written for one platform will work on any other platform that has the Python interpreter installed. (This is true of any interpreted language, not just Python.)

Python is Simple

As programming languages go, Python is relatively uncluttered, and the developers have deliberately kept it that way. A rough estimate of the complexity of a language can be gleaned from the number of keywords or reserved words in the language. These are words that are reserved for special meaning by the compiler or interpreter because they designate specific built-in functionality of the language.

FLASK

Flask is a lightweight and versatile web framework for building web applications in Python. Its minimalist design provides the core functionalities needed for web development without imposing unnecessary complexity, making it ideal for small to medium-sized projects. Key features of Flask

include routing, templating with Jinja2, HTTP request handling, session management, and a built-in development server for easy testing during development.

One of Flask's strengths lies in its flexibility and ease of use. Developers have the freedom to customize their applications as per their specific requirements, and its simplicity makes it a great choice for both beginners and experienced developers. Flask's URL routing system allows developers to map specific URL patterns to functions, making navigation and handling different HTTP methods straightforward.

The templating engine, Jinja2, enables developers to separate logic and presentation, facilitating dynamic rendering of HTML content and displaying data from the server. Moreover, Flask supports session management, allowing developers to store user-specific data across multiple requests.

With Flask, developers can create web applications and RESTful APIs with ease, making it a popular choice for a wide range of use cases. Additionally, its ecosystem of extensions provides a wealth of additional functionalities, including database integration, authentication, and security features.

Flask's active community, extensive documentation, and continuous development ensure that it remains up-to-date and well-supported. Overall, Flask's simplicity, flexibility, and ease of learning have made it a go-to web framework for many Python developers seeking to build efficient and reliable web applications.

HTML

HTML (Hyper Text Markup Language) is the most basic building block of the Web. It defines the meaning and structure of web content. Other technologies besides HTML are generally used to describe a web page's appearance/presentation (CSS) or functionality/behavior (JavaScript). "Hypertext" refers to links that connect web pages to one another, either within a single website or between websites. Links are a fundamental aspect of the Web. By uploading content to the Internet and linking it to pages created by other people, you become an active participant in the World Wide Web. It is the standard markup language used for creating web pages and web applications. HTML forms the backbone of most websites, providing the basic structure upon which CSS and JavaScript are applied to manipulate appearance and functionality.

Structure: HTML uses tags to define elements such as headings, paragraphs, links, images, and lists. These elements structure the content on a webpage.

➤ HTML Headings:

Use the heading tags to show the structure and importance of the content on a page. Always use the h1 tag to identify the most important information on the page, and only code a single h1 tag on each page. Then, decrease one level at a time to show subsequent levels of importance.

```
<h1>Welcome to HTML</h1>
```

➤ HTML Paragraphs:

The element defines a paragraph. It uses to deal with blocks of text in the web site document. if we add another element such as

```
<p>Paragraph tag</p>
```

➤ HTML link:

It generally provides additional styling (attribute) to the element.

Attributes appear inside the opening tag and their values sit inside quotation marks.

```
</img>
```

Semantics: Semantic tags describe the meaning and structure of web content beyond mere presentation. Examples include `<article>`, `<section>`, `<nav>`, and `<footer>`.

Embedding Media: HTML supports embedding images, audio, and video into a web page using the `<img>`, `<audio>`, and `<video>` tags, respectively.

Forms: HTML provides elements for creating forms to collect user input, including `<input>`,

`<textarea>`, and `<button>`, along with form controls like checkboxes and radio buttons.

- Use semantic markup for better accessibility and SEO.
- Keep your code clean and well-organized for easier maintenance.
- Ensure your HTML is valid and follows web standards.

HTML DOCUMENT STRUCTURE

HTML document contains the text (the content of the page) with embedded tags, which provide instruction, appearance and function of the content. The HTML document is divided into two major portions: the head and the body.

- ✓ The head contains information about the document such as the title and "meta" information describing the content.
- ✓ The body contains the actual contents of the document (the parts that is displayed in the browser window).

CSS (CASCADING STYLE SHEETS)

CSS (Cascading Style Sheets) is a powerful design language used to define the visual presentation of documents written in HTML or XML. It plays a crucial role in modern web development by separating the content of a webpage from its appearance, enabling developers to control layout, colors, fonts, and other stylistic elements. With CSS, designers can make web pages more attractive, interactive, and responsive across various devices.

The primary purpose of CSS is to enhance the look and feel of a webpage. It allows customization of text color, font styles, spacing between paragraphs, layout of columns, background images or colors, and many other visual effects. By applying CSS rules to HTML elements, web developers can ensure consistency in design, improve user experience, and reduce code duplication.

CSS follows a structured syntax where each rule consists of a selector, a property, and a value. The selector targets the HTML element to be styled, the property specifies the aspect to be changed (such as color or margin), and the value assigns a specific setting to that property. For example, using CSS, a paragraph can be styled with a red font color and a specific font size, contributing to the visual appeal of the page.

Beyond layout, CSS is instrumental in **styling elements** with fonts, background colors, images, gradients, borders, and shadows. It also supports **animations and transitions**, adding dynamic effects to elements when users interact with them or when changes occur on the page. This enhances interactivity and creates a more engaging user experience.

CSS is typically written in **external style sheets**, which promote better code organization and performance. They allow styles to be reused across multiple pages and enable the browser to cache them for faster loading times.

Additionally, **responsive design** is a key advantage of CSS. Through media queries and flexible units, CSS ensures that websites adapt seamlessly to different screen sizes and devices, from desktops to smartphones.

Some best practices in CSS development include using clear and semantic class names, keeping code DRY (Don't Repeat Yourself), organizing styles logically, and commenting generously for readability. Proper use of CSS contributes to well-structured, maintainable, and visually appealing web applications.

Cascading Style Sheets, fondly referred to as CSS, is a simple design language intended to simplify the process of making web pages presentable.

CSS handles the look and feel part of a web page. Using CSS, you can control the color of the text, the style of fonts, the spacing between paragraphs, how columns are sized and laid out, what background images or colors are used, as well as a variety of other effects.

It is a style sheet language used for describing the presentation of a document written in HTML or XML (including XML dialects like SVG or XHTML). CSS describes how elements should be rendered on screen, on paper, in speech, or on other media.

CSS SYNTAX

A CSS comprises of style rules that are interpreted by the browser and then applied to the corresponding elements in your document. A style rule is made of three parts:

- ✓ Selector: A selector is an HTML tag at which a style will be applied. This could be any tag like `<h1>` or `<table>` etc.
- ✓ Property: A property is a type of attribute of HTML tag. Put simply, all the HTML attributes are converted into CSS properties. They could be color, border, etc.

- ✓ Value: Values are assigned to properties. For example, color property can have the value either red or #F1F1F1 etc.

```
Selector {  
    Property: value;  
}
```

Core Features

- Selectors: CSS uses selectors to apply styles to elements and groups of elements. Selectors can target elements by tag name, class, id, and more.
- Box Model: Every element in CSS has a box model, consisting of margins, borders, padding, and the actual content. Understanding this model is crucial for layout design.
- Layouts: CSS provides various layout techniques such as Flex box and Grid, enabling responsive and flexible layouts.
- Styling: Beyond layout, CSS controls fonts, colors, backgrounds, transitions, and animations, allowing for richly styled web pages.
- Use external style sheets for maintainability and performance.
- Employ responsive design principles to ensure your site looks good on all devices. Organize your CSS logically and comment extensively for clarity.

CSS Properties

Control many style properties of an element:

- Coloring
- Size
- Position
- Visibility

- Many more: (e.g. p: { text-decoration: line-through; })
- Also used in animation

JAVASCRIPT

JavaScript is a lightweight, interpreted programming language. It is designed for creating network-centric applications. It is complimentary to and integrated with Java. JavaScript is very easy to implement because it is integrated with HTML. It is open and cross-platform.

JavaScript is a programming language that enables interactive web pages. It is an essential part of web applications, allowing for client-side script to interact with the user, control the browser, communicate asynchronously, and alter document content that is displayed.

- ✓ JavaScript is a lightweight, interpreted programming language.
- ✓ Designed for creating network-centric applications.
- ✓ Complementary to and integrated with Java.
- ✓ Complementary to and integrated with HTML.
- ✓ Open and cross-platform.

Core Feature

Interactivity: JavaScript allows users to interact with web pages. Common examples include form validation, pop-up messages, and dynamic content updates.

- DOM Manipulation: JavaScript can manipulate the Document Object Model (DOM), allowing scripts to change document content, style, and structure dynamically.
- Events: JavaScript can respond to user actions such as clicks, form submissions, and page loads, making web pages more responsive and interactive.

- Frameworks and Libraries: There are numerous JavaScript frameworks and libraries (e.g., React, Angular, and Vue.js) that provide pre-written code to help develop robust and high-performance web applications.
- Keep code modular and use functions to avoid repetition.
- Ensure compatibility with different browsers and devices.
- Focus on security, especially when dealing with user input and data

manipulation. Each of these technologies plays a distinct role in web development, with HTML providing the basic structure, CSS adding styling to make the web page visually appealing, and JavaScript adding interactivity to enhance user experience.

DISADVANTAGES OF JAVASCRIPT

- Less server interaction: You can validate user input before sending the page off to the server. This saves server traffic, which means less load on your server.
- Immediate feedback to the visitors: They don't have to wait for a page reload to see if they have forgotten to enter something.
- Increased interactivity: You can create interfaces that react when the user hovers over them with a mouse or activates them via the keyboard.
- Richer interfaces: You can use JavaScript to include such items as drag-and-drop components and sliders to give a Rich Interface to your site visitors.

JAVASCRIPT – SYNTAX

- ✓ JavaScript can be implemented using JavaScript statements that are placed within the `<script>...</script>` HTML tags in a web page.
- ✓ You can place the `<script>` tags, containing your JavaScript, anywhere within your web page, but it is normally recommended that you should keep it within the `<head>` tags.

- ✓ The <script> tag alerts the browser program to start interpreting all the text between these tags as a script. A simple syntax of your JavaScript will appear as follows.

```
<script language="javascript"
      type="text/javascript"> Javascript
      code</script>
```

5.2 MODULE DESCRIPTION

5.2.1 DATA COLLECTION MODULE

The Data Collection Module is responsible for gathering software metrics required for analysis. It collects important input features such as CBO, fanin, fanout, WMC, LOC, and other complexity-related metrics. These values are usually taken from datasets or software repositories. The collected data serves as the foundation for the entire system. It ensures that sufficient and relevant data is available for model training. Accurate data collection improves the overall performance of the prediction system

The data collection module ensures that all required software metrics are gathered accurately from reliable sources. It provides the necessary input data for the system and forms the foundation for model training and prediction.

5.2.2 DATA PREPROCESSING MODULE

The Data Preprocessing Module prepares the collected data for machine learning. It removes unwanted or non-numeric values from the dataset. Missing values and inconsistencies are handled to ensure clean data. Important features are selected to improve prediction accuracy. Data is structured properly so that it can be easily used for training. This step ensures the model receives high-quality input data.

The preprocessing module refines the collected data by removing errors and organizing it into a structured format. This improves data quality and ensures better performance of the machine learning model.

5.2.3 RISK LABEL CREATION MODULE

This module assigns risk levels to the dataset based on predefined conditions. It categorizes data into Low, Medium, and High risk levels. For example, risk can be assigned based on LOC or complexity values. These labels act as the target output for the machine learning model. It helps the model understand patterns related to risk levels. Proper labeling is essential for accurate predictions.

This module assigns appropriate risk levels to the dataset based on predefined conditions. Proper labeling helps the model learn patterns effectively and improves prediction accuracy.

5.2.4 MACHINE LEARNING MODEL MODULE

This module is the core part of the system where the model is trained. It uses the Random Forest algorithm to learn from the dataset. The model identifies patterns between input metrics and risk levels. Training and testing processes are carried out to evaluate performance. The model improves its accuracy based on the training data. Once trained, it is ready to predict new inputs. The machine learning module builds and trains the prediction model using the processed data. It identifies relationships between input metrics and risk levels to generate accurate predictions.

5.2.5 PREDICTION MODULE

The Prediction Module is used to predict the risk level for new data. It takes user input values such as CBO, WMC, LOC, and other metrics. These inputs are passed to the trained machine learning model. The model then predicts whether the software is Low, Medium, or High risk. The output is generated instantly for the user. This module provides the final result of the system. The prediction module uses the trained model to classify new input data into risk categories. It provides quick and reliable results, helping users make informed decisions.

5.2.6 WEB INTERFACE MODULE

The Web Interface Module provides a user-friendly platform to interact with the system. It is developed using Flask and HTML. Users can enter software metrics through input forms. The system displays prediction results and graphs on the screen. It makes the application easy to use without technical knowledge. This module enhances the overall usability and presentation of the project. The web interface module enables user interaction with the system through a simple and intuitive platform. It allows users to input data and view prediction results easily.

CHAPTER – 6

METHODOLOGY

The methodology of this project involves several systematic steps to predict software risk using machine learning. Initially, the dataset containing software metrics is collected and loaded into the system. In the preprocessing stage, non-numeric values are removed and the data is cleaned for better accuracy. Important features such as coupling, cohesion, and complexity metrics are selected for analysis. A risk label is then created by categorizing data into Low, Medium, and High risk levels based on predefined conditions. The processed data is divided into training and testing sets. The Random Forest algorithm is used to train the model using the training data. After training, the model is evaluated using performance metrics like accuracy and recall. The trained model is then used to predict risk levels for new input data. Finally, the results are displayed through a web interface along with graphical visualization for better understanding.

6.1 DATA COLLECTION

The data collection process involves gathering software metrics required for analysis and prediction. The dataset includes important features such as cbo, fanin, fanout, wmc, loc, and other code-related metrics. These values are collected from software repositories or existing datasets and stored in a structured format like CSV. The collected data acts as the foundation for training the machine learning model. Accurate and relevant data collection is essential to ensure better prediction results. Without proper data, the performance of the system will be affected.

6.2 DATA PREPROCESSING

Data preprocessing is an important step that prepares the collected data for machine learning. In this stage, non-numeric values are removed, and missing or incorrect values are handled properly. The dataset is cleaned and formatted so that it can be used efficiently for model training. This process ensures that the data is consistent and error-free. Proper preprocessing helps in improving the accuracy and performance of the model. It also reduces noise in the dataset.

6.3 FEATURE SELECTION

Feature selection involves choosing the most important metrics from the dataset for analysis. Not all features contribute equally to prediction, so irrelevant or less important features are removed. Important metrics such as cbo, wmc, loc, and lcom are selected for the model. This helps reduce complexity and improves model efficiency. By selecting only relevant features, the model can learn faster and provide better results. It also helps in avoiding overfitting.

6.4 RISK LABEL CREATION

In this step, the dataset is labeled with risk levels based on predefined conditions. The data is classified into three categories: Low Risk, Medium Risk, and High Risk. For example, risk levels can be assigned based on the LOC values or other complexity metrics. These labels act as the target output for the machine learning model. This step converts the dataset into a classification problem. Proper labeling is crucial for accurate prediction

6.5 MODEL TRAINING

Model training is the core part of the system where the machine learning algorithm learns from the data. In this project, the Random Forest algorithm is used to train the model. The model learns patterns and relationships between input features and risk levels. Multiple decision trees are created, and their results are combined to improve accuracy. The training process helps the model understand how to predict risk for new data. A well-trained model gives better performance.

6.6 MODEL TESTING AND EVALUATION

After training, the model is tested using a separate dataset to evaluate its performance. The model predictions are compared with actual values to measure accuracy. Metrics such as accuracy, recall, and confusion matrix are used for evaluation. This step helps to identify how well the model is performing. If needed, improvements can be made to increase accuracy. Proper evaluation ensures the reliability of the model.

6.7 PREDICTION

The prediction stage is where the trained model is used to predict risk levels for new input data. The user provides software metrics through the system interface. These inputs are processed by the model to generate a prediction. The output is classified as Low Risk, Medium Risk, or High Risk. The result is generated instantly for the user. This is the final outcome of the system.

6.8 VISUALIZATION

Visualization is used to display the prediction results in a graphical format. Bar charts or line graphs are used to show the probability of each risk level. This helps users easily understand the results. Visual representation makes the output more clear and interactive. It improves user experience and supports better decision-making. Graphs make the system more attractive and informative.

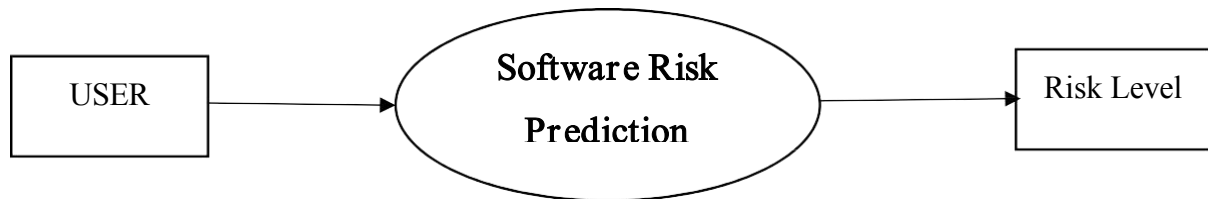
6.9 WEB INTERFACE

The web interface provides a platform for users to interact with the system. It is developed using Flask along with HTML and CSS. Users can enter software metrics through a form and submit the data. The system processes the input and displays the prediction result along with graphs. The interface is designed to be simple and user-friendly. It allows even non-technical users to use the system easily

6.10 SYSTEM DIAGRAMS

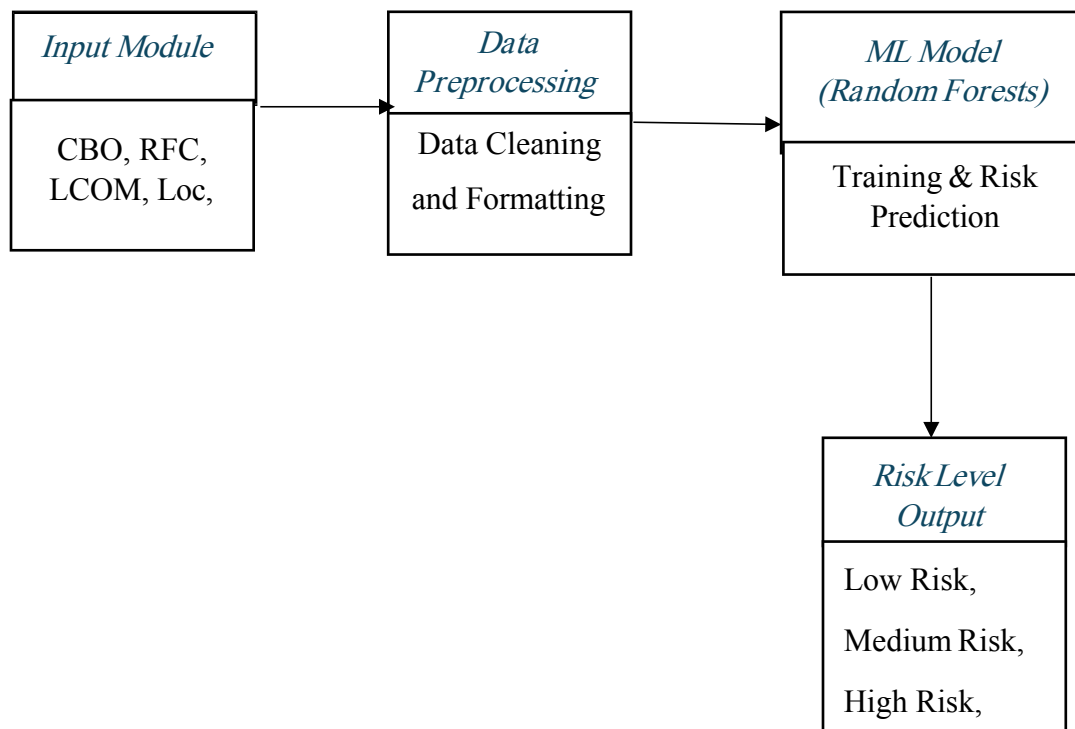
6.10.1 DATA FLOW DIAGRAM (DFD)

LEVEL 0



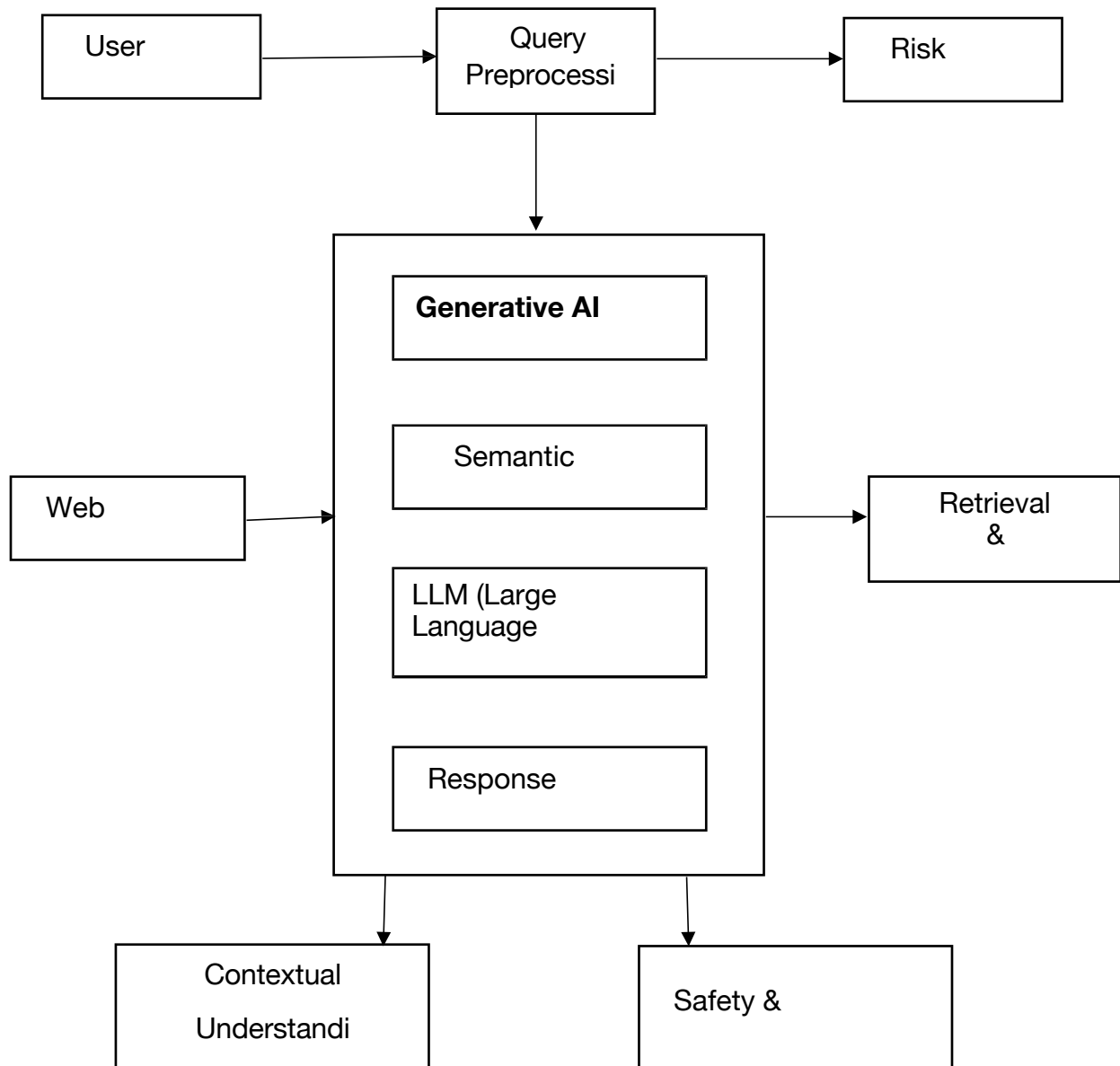
6.10.1 DFD LEVEL 0

6.10.2 DFD LEVEL 1



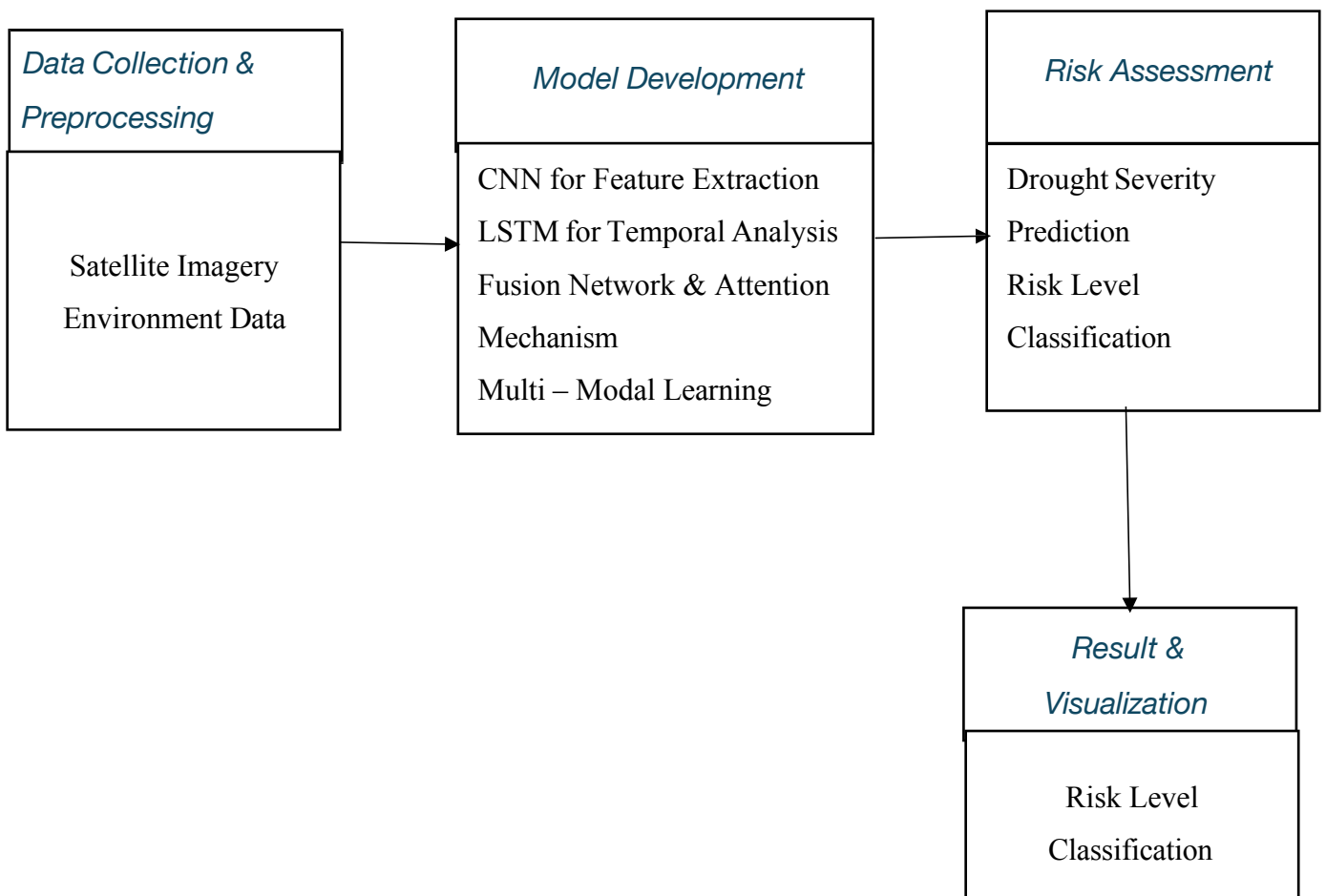
6.10.2 DFD LEVEL 1

6.10.3 SYSTEM ARCHITECTURE DIAGRAM



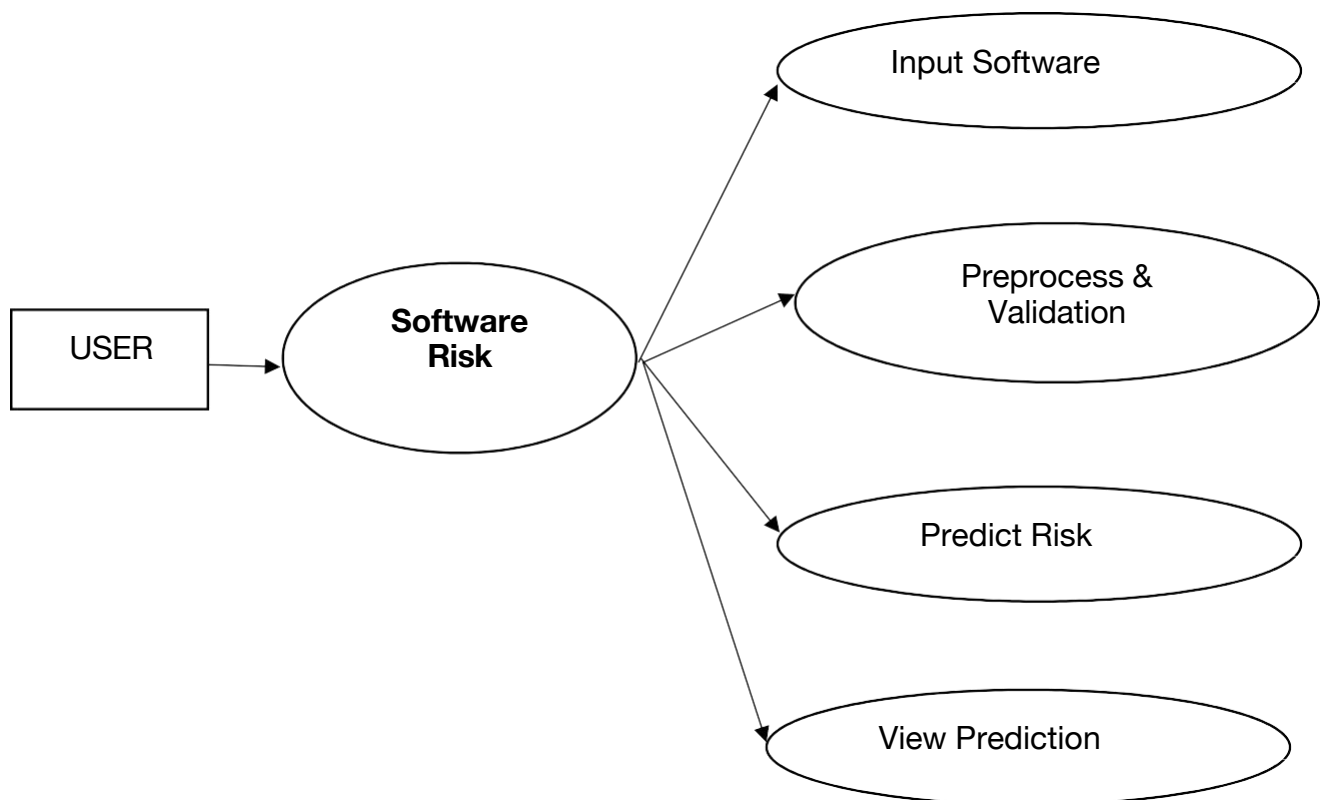
6.10.3 SYSTEM ARCHITECTURE DIAGRAM

6.10.4 METHODOLOGY DIAGRAM



6.10.4 METHODOLOGY DIAGRAM

6.10.5 USECASE DIAGRAM



6.10.5 USECASE DIAGRAM

6.11 RANDOM FOREST ALGORITHM

Random Forest is a popular machine learning algorithm used for classification and regression tasks. It is an ensemble learning method that combines multiple decision trees to improve accuracy and performance. Instead of relying on a single decision tree, Random Forest builds many trees and merges their results. Each tree is trained using a random subset of the dataset and features, which increases diversity among the trees. This randomness helps in reducing overfitting and improves generalization. During prediction, each tree gives its output, and the final result is decided by majority voting in classification problems.

In regression tasks, the average of all tree outputs is taken. Random Forest is highly effective in handling large datasets with many features. It can also manage missing values and maintain accuracy. The algorithm provides feature importance, which helps in identifying the most influential variables.

It is robust and less sensitive to noise in the data. Random Forest works well for complex problems where relationships between variables are not linear. It is widely used in real-world applications like risk prediction, fraud detection, and medical diagnosis. In this project, Random Forest is used to predict software risk levels based on code metrics. Overall, it is a powerful and reliable algorithm for accurate predictions.

Furthermore, Random Forest uses a technique called bagging (bootstrap aggregating), where multiple subsets of data are created with replacement to train individual trees. This improves the stability and accuracy of the model. The algorithm also selects a random subset of features at each split, which ensures that no single feature dominates the model. This combination of randomness in both data and feature selection makes Random Forest highly reliable and reduces the chances of model bias.

CHAPTER – 7

SYSTEM TESTING AND IMPLEMENTATION

7.1 SYSTEM TESTING

System testing is an important phase in the development of this project to ensure that the system works correctly and efficiently. In this stage, the entire application is tested as a whole to verify its functionality.

The input values entered by the user are checked to ensure proper validation and handling. The machine learning model is tested using different sets of data to evaluate its prediction accuracy.

The system is also tested for errors, bugs, and unexpected behavior. The web interface is verified to ensure smooth navigation between pages such as Home, Predict, and Result. The prediction results and graphs are checked for correctness and proper display.

Performance testing is done to ensure the system responds quickly to user inputs. Compatibility testing ensures the system works properly on different browsers and environments. Overall, system testing ensures that the application is reliable, accurate, and ready for real-world usage.

7.1.1 UNIT TESTING

Unit testing is performed to check the individual components of the system. Each module such as data loading, preprocessing, model training, and prediction is tested separately. This ensures that every part of the system works correctly without errors. Bugs are identified and fixed at an early stage. It helps in improving the reliability of each module. Unit testing makes the overall system stable.

Furthermore, unit testing ensures that each module handles edge cases and unexpected inputs correctly. Test cases are designed for different scenarios to validate module behavior.

Automated testing tools can also be used to repeatedly test individual components. This reduces manual effort and improves testing efficiency. Continuous unit testing helps maintain code quality during development. It also simplifies debugging by isolating issues within specific modules.

7.1.2 INTEGRATION TESTING

Integration testing is used to verify the interaction between different modules. It ensures that modules like preprocessing, model training, and prediction work together properly. Data flow between modules is checked to avoid errors. This testing confirms that combined components function as expected. It helps in identifying interface issues between modules. Proper integration improves system performance. In addition, integration testing verifies the consistency of data exchange between modules. It ensures that the output of one module is correctly accepted as input by another. Different integration strategies such as incremental testing can be applied. Errors related to data mismatch or communication failure are identified and resolved. This improves coordination between system components. Overall, it ensures smooth workflow across the entire system.

7.1.3 SYSTEM TESTING

System testing checks the complete system as a whole. The entire application, including the web interface and machine learning model, is tested together. It ensures that all functionalities are working correctly. User inputs, predictions, and output display are verified. This testing ensures that the system meets the project requirements. It validates the overall performance of the system.

Moreover, system testing evaluates the system under real-world conditions to ensure reliability. Different user scenarios are tested to validate system behavior. Both functional and non-functional aspects are examined during this phase. This testing ensures that the system operates without failure in a complete environment. It also helps in identifying any hidden issues. Overall, it confirms that the system is ready for deployment.

7.1.4 FUNCTIONAL TESTING

Functional testing verifies whether the system performs according to its intended functions. It checks features such as input submission, prediction generation, and result display. Each function is tested with valid and invalid inputs. The system's response is observed for correctness. This ensures that all features are working as expected. It helps in delivering a complete functional system. Additionally, functional testing ensures that all user actions produce the expected outcomes. Test cases are created based on system requirements and user scenarios. The system is tested with a variety of inputs to check accuracy and consistency. Any deviation from expected results is identified and corrected. This testing guarantees that all features perform correctly. It ensures the system fulfills its intended purpose.

7.1.5 PERFORMANCE TESTING

Performance testing evaluates how fast and efficiently the system responds. It checks the response time of prediction and graph generation. The system is tested under different input conditions. This ensures that the application does not slow down or crash. It helps in improving speed and efficiency. Good performance enhances user experience.

Furthermore, performance testing evaluates system behavior under load conditions. The system is tested with multiple inputs to check stability and

response time. It helps in identifying bottlenecks and optimizing system performance. Memory usage and processing time are also analyzed. This ensures that the application performs efficiently even under heavy usage. It contributes to a smooth and reliable user experience.

7.1.6 USER INTERFACE TESTING

User interface testing ensures that the web pages are user-friendly and easy to use. It checks the design, layout, and navigation of the application. Buttons, forms, and links are tested for proper functionality. The display of results and graphs is verified. This ensures a smooth interaction for users. A good UI improves usability and satisfaction.

In addition, user interface testing ensures consistency in design and responsiveness across different devices and browsers. It checks alignment, readability, and accessibility of elements. User feedback can also be considered to improve interface design. This testing ensures that users can easily interact with the system without confusion. It enhances usability and overall user satisfaction. A well-tested UI improves the effectiveness of the application.

7.1.7 VALIDATION TESTING

Validation testing ensures that the system meets user requirements. It checks whether the final output matches the expected results. The prediction accuracy is verified using test data. This confirms that the system solves the intended problem. It ensures that the model gives meaningful predictions. Validation testing confirms the success of the project.

Moreover, validation testing involves comparing system outputs with actual or expected results using test datasets. Statistical measures such as accuracy and error rates can be analyzed. This testing ensures that the model performs well on unseen data. It confirms the reliability and correctness of

predictions. Continuous validation helps improve model performance over time. It plays a key role in ensuring the success of the system.

7.1.8 ERROR TESTING

Error testing is performed to check how the system handles invalid inputs. Incorrect or missing values are given as input. The system response is observed to ensure proper error handling. It prevents system crashes and unexpected behavior. Proper error messages are displayed to the user. This improves system robustness and reliability.

Furthermore, error testing ensures that the system can handle boundary and extreme values effectively. Inputs such as very large numbers, negative values, and zero values are tested to verify system stability. The system is designed to validate inputs before processing, ensuring that incorrect data does not affect prediction results. Proper exception handling mechanisms are implemented to manage runtime errors gracefully. Logging of errors is also maintained to help in debugging and future improvements. User-friendly error messages guide users to correct their inputs without confusion. This testing helps in identifying potential weaknesses in the system and improving overall performance. Thus, error testing plays a crucial role in ensuring a stable, reliable, and user-friendly application.

7.2 SYSTEM IMPLEMENTATION

The system implementation of this project involves developing a complete software risk prediction system using machine learning techniques and a web-based interface. Initially, the dataset containing software metrics is collected and loaded into the system using Python. The preprocessing stage is implemented to clean the data by removing non-numeric values and handling missing data.

Important features such as cbo, fanin, fanout, wmc, loc, and lcom are selected for analysis. A risk label is then created by categorizing the data into Low, Medium, and High risk levels based on predefined conditions. The dataset is divided into training and testing sets to build and evaluate the model. The Random Forest algorithm is implemented to train the model, as it provides high accuracy and handles complex data effectively.

The trained model learns patterns between input features and risk levels. After training, the model is tested using test data to evaluate its performance. Metrics such as accuracy, recall, and confusion matrix are used to measure the effectiveness of the model. Once the model achieves satisfactory performance, it is saved using joblib for future predictions. This completes the machine learning implementation part of the system.

The second part of the implementation focuses on developing the web-based application using Flask. A user-friendly interface is created with multiple pages such as Home, About, Predict, and Result.

The Predict page allows users to enter software metrics through input forms. When the user submits the form, the input data is sent to the Flask backend. The backend processes the input, loads the trained model, and performs prediction. The predicted risk level is generated as Low, Medium, or High. The result is then displayed on the Result page along with graphical visualization using Chart.js. The graph shows the probability of each risk level, making the output easy to understand.

Navigation between pages is implemented using a navbar for better user experience. The system is tested to ensure smooth functionality and correct output. Error handling is also implemented to manage invalid inputs. The final system provides a complete solution for software risk prediction.

Overall, the implementation integrates machine learning with web technology to create an efficient and user-friendly application. Furthermore, the system is designed to be scalable and efficient, allowing it to handle multiple user requests without affecting performance. The Flask backend ensures smooth communication between the user interface and the machine learning model.

The application structure is organized in a modular way, making it easier to update or extend functionalities in the future. This design approach improves maintainability and supports continuous development.

In addition, security and data validation are given importance to ensure safe system operation. User inputs are validated before processing to prevent incorrect or harmful data from affecting the model. The system also ensures quick response time, providing results instantly after submission.

This enhances user experience and makes the application suitable for real-world usage. Overall, the implementation ensures reliability, scalability, and ease of use.

CHAPTER – 8

SYSTEM MAINTENANCE

8.1 OVERVIEW

System maintenance is an essential phase that ensures the continuous and efficient functioning of the developed software risk prediction system. After the deployment of the system, regular monitoring is required to identify and fix any errors or bugs that may arise during usage. Maintenance activities include updating the machine learning model with new data to improve prediction accuracy over time. As software projects evolve, new metrics and features can be added to enhance the system's capability.

The dataset may need to be updated periodically to reflect real-world changes in software development practices. Performance tuning is also carried out to ensure that the system responds quickly to user inputs. In addition, security updates are necessary to protect the system from potential threats and vulnerabilities. The web interface may require design improvements to enhance user experience. Compatibility with different browsers and devices is also maintained.

Backup and recovery mechanisms are implemented to prevent data loss. Logging and monitoring tools can be used to track system performance and detect issues early. Proper documentation is maintained to support future updates and modifications. Overall, system maintenance ensures that the application remains reliable, secure, and efficient over time.

Another important aspect of system maintenance is adaptability and scalability. The system should be flexible enough to incorporate new

technologies and algorithms as they become available. For example, more advanced machine learning models such as XGBoost or deep learning techniques can be integrated in the future.

The system can also be extended to support real-time data processing and cloud- based deployment. Continuous improvement is achieved by collecting user feedback and making necessary enhancements. Regular testing is performed after updates to ensure that existing functionalities are not affected. Maintenance also includes optimizing the database and improving data handling efficiency. The system should be able to handle increasing amounts of data as usage grows. Updating dependencies and software libraries is necessary to maintain compatibility and performance.

Error handling mechanisms are improved to provide better user feedback. Training the model with updated datasets helps in maintaining prediction accuracy. The interface can be enhanced with advanced visualizations and dashboards. Overall, system maintenance plays a vital role in ensuring long-term usability, scalability, and effectiveness of the software risk prediction system.

8.2 TYPES OF MAINTENANCE

Corrective Maintenance

Corrective maintenance focuses on identifying and fixing bugs, errors, and defects that are discovered after the system has been deployed. These issues may arise due to unexpected user inputs, system failures, or overlooked problems during testing.

The goal is to restore the system to its normal functioning state as quickly as possible. This type of maintenance ensures system reliability and prevents further issues from affecting users.

Adaptive Maintenance

Adaptive maintenance involves modifying the system to keep it compatible with changing environments such as new operating systems, hardware updates, or software dependencies.

As technology evolves, systems must adapt to continue functioning properly. This includes updating libraries, frameworks, and configurations. It ensures that the system remains usable and effective in different and changing technological conditions.

Perfective Maintenance

Perfective maintenance is carried out to enhance the performance, usability, and overall efficiency of the system. It includes improving existing features, optimizing system

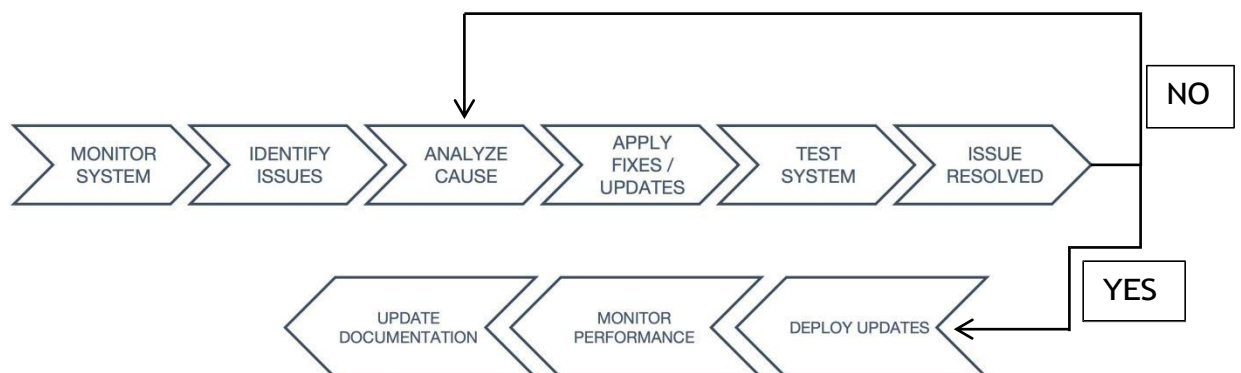
processes, and adding new functionalities based on user feedback. This type of maintenance helps in making the system more user-friendly and efficient. It contributes to better user satisfaction and improved system quality over time.

Preventive Maintenance

Preventive maintenance aims to identify and resolve potential issues before they become major problems. It includes activities such as code refactoring, performance tuning, security updates, and system monitoring.

By taking proactive measures, the system's stability and reliability are improved. This helps in reducing future maintenance costs and avoiding unexpected system failures.

8.3 WORKFLOW



8.3WORKFLOW DIAGRAM

CHAPTER - 9

SYSTEM DESIGN

9.1 INPUT DESIGN

Input design is an important part of the system that focuses on how user data is collected and processed efficiently. In this project, the input design is developed to accept software metrics required for risk prediction.

The system provides a user-friendly web interface where users can enter values such as cbo, fanin, fanout, wmc, dit, noc, rfc, lcom, and loc. Additional metrics like loop quantity, comparisons quantity, and maximum nested blocks are also included. All input fields are designed using form elements to ensure easy data entry. Proper validation is implemented to ensure that only numeric values are accepted.

This helps in avoiding errors during prediction. Default values or placeholders are provided to guide users while entering data. The input design ensures that all required fields are filled before submission. Data consistency is maintained by restricting invalid or missing inputs. The entered data is then converted into a structured format suitable for the machine learning model. The system processes the input quickly and efficiently.

User-friendly labels are used to clearly describe each input field. The design minimizes user effort and reduces confusion. Overall, the input design ensures accurate data collection and smooth interaction with the system.

Furthermore, the input design is structured to ensure scalability and flexibility for future enhancements. The system can be extended to include additional software metrics without affecting the existing functionality. Input data is preprocessed before being passed to the machine learning model, which includes normalization and formatting of values.

This ensures that the model receives consistent and standardized inputs

for accurate prediction. The interface is designed to handle multiple user inputs efficiently, making it suitable for repeated usage. Error messages are displayed clearly if invalid data is entered, guiding the user to correct the input. The design also considers responsiveness, allowing the system to function properly across different devices and screen sizes. Overall, the input design is robust, flexible, and capable of supporting future system improvements.

Moreover, the input design incorporates validation mechanisms at both the client- side and server-side to ensure higher data accuracy and security. Client-side validation provides immediate feedback to users, while server-side validation ensures that incorrect or manipulated data does not affect the system.

This dual-layer validation improves reliability and prevents invalid data from entering the prediction process. It also enhances system robustness by handling unexpected inputs effectively.

Another important aspect of the input design is its support for extensibility and integration. The system can be connected with external data sources such as software repositories or APIs to automatically fetch software metrics. This reduces manual data entry and increases efficiency. The input module can also be integrated with continuous development tools, enabling automated data collection in future enhancements. This makes the system more suitable for real-world industrial application.

9.2 OUTPUT DESIGN

Output design is an important aspect of the system that focuses on how the results are presented to the user in a clear and understandable manner. In this project, the output is designed to display the predicted software risk level based on the given input metrics.

The system provides the result in three categories: Low Risk, Medium Risk, and High Risk. The output is displayed on a dedicated result page after the

user submits the input values. The design ensures that the prediction is clearly highlighted for easy understanding. In addition to the textual result, the system also provides graphical visualization using charts.

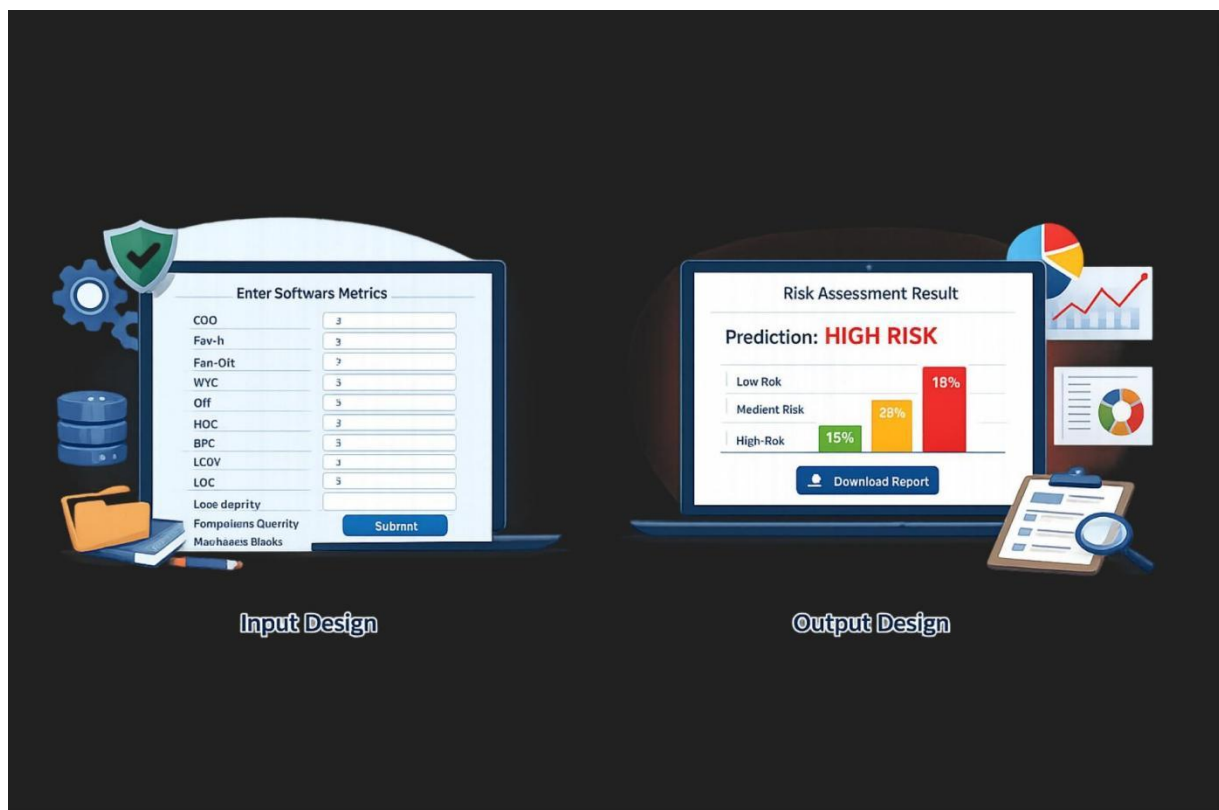
A bar graph is used to show the probability of each risk level. This helps users understand how the prediction is derived. The output page is designed using Bootstrap to provide a clean and professional look. Colors are used to differentiate risk levels, such as green for low, yellow for medium, and red for high risk.

The system ensures that the output is generated quickly and accurately. Navigation options are provided to return to the input page for new predictions. The design focuses on clarity, simplicity, and user satisfaction. Overall, the output design enhances user experience by presenting results in an effective and visually appealing way.

In addition, the output design is developed to provide detailed insights into the prediction results. Along with the risk classification, the system can display probability scores for each category, helping users better understand the confidence level of the prediction. The graphical representation enhances interpretability by presenting data in an intuitive format. The system also ensures fast response time, allowing users to receive results instantly after submission. The output page can be extended to include downloadable reports for documentation and analysis purposes. Moreover, the design supports easy integration with dashboards for monitoring multiple predictions.

The overall structure ensures that the output is not only informative but also actionable for developers and project managers. Thus, the output design plays a crucial role in improving usability and decision-making efficiency. Furthermore, the output design ensures consistency and clarity in presenting results across different user sessions. Standard formatting and structured layouts are

maintained to make the output easy to interpret. The system highlights key results prominently so that users can quickly identify the predicted risk level. This improves readability and reduces the chances of misinterpretation. Another enhancement in output design is the possibility of incorporating comparative analysis.



9.1 INPUT AND OUTPUT DESIGN

This helps users understand trends and variations in software quality over time. Such features are useful for project monitoring and continuous improvement. Additionally, the output design can support export and reporting features, allowing users to download results in formats such as PDF or CSV. This is useful for documentation, analysis, and sharing with team members or stakeholders. Integration with reporting tools can further enhance usability and make the system more practical for professional environments.

CONCLUSION

In conclusion, this project presents an effective approach for software quality analytics and risk prediction using machine learning techniques. The system successfully analyzes important software metrics such as coupling, cohesion, complexity, and code size to determine the risk level of software modules. By using the Random Forest algorithm, the model is able to accurately classify the software into Low, Medium, and High risk categories. This helps developers identify potential problem areas in the early stages of development. The project addresses the limitations of traditional methods by providing an automated and intelligent solution.

It reduces manual effort and improves the efficiency of software quality assessment. The integration of a web-based interface using Flask makes the system user-friendly and accessible. Users can easily input software metrics and obtain instant predictions. The graphical visualization of prediction results enhances understanding and decision-making. The system also demonstrates good performance in terms of accuracy and reliability. It can handle multiple input features and provide consistent results. The project contributes to improving software reliability and reducing development costs. It also supports better planning and resource allocation for software projects. The modular design of the system allows easy extension and improvement.

The implementation proves that machine learning can be effectively applied in software engineering. The system is flexible and can be adapted to different types of software projects. It also provides a foundation for future research and development. Overall, the project achieves its objective of predicting software risks efficiently.

It highlights the importance of early risk detection in maintaining software quality. The combination of machine learning and web technology

makes the system practical and scalable. Hence, this project offers a valuable solution for modern software development and risk management.

In addition to the above, the proposed system further enhances software development by enabling data-driven decision making and reducing reliance on manual judgment. It improves testing prioritization by allowing developers to focus on high-risk modules, thereby optimizing resource utilization and development effort.

The approach supports early detection of defects, which helps in minimizing costly fixes at later stages of the software lifecycle. The model can be continuously retrained with updated datasets to maintain and improve its prediction accuracy over time.

Moreover, the system demonstrates scalability and can be effectively applied to large- scale software projects. It can also be integrated with CI/CD pipelines for continuous monitoring of software quality. Overall, the solution promotes automation in software engineering practices, reduces failure rates after deployment, and contributes to the development of more reliable, maintainable, and efficient software systems.

Furthermore, the proposed system highlights the growing importance of intelligent automation in modern software engineering practices. By leveraging machine learning techniques, the system transforms raw software metrics into meaningful insights that support proactive decision-making.

This shift from reactive debugging to predictive analysis significantly improves the overall development process and reduces uncertainty in project outcomes. It also encourages the adoption of data-driven methodologies in software quality management.

In addition, the system promotes a structured approach to software risk assessment by providing consistent and objective results. Unlike traditional

manual evaluation methods, the model ensures uniformity in predictions across different projects. This helps organizations maintain standard quality levels and improve development efficiency. The ability to visualize risk levels further enhances clarity, making it easier for stakeholders to interpret and act upon the results.

Another important contribution of the project is its potential for integration into real-world development environments. The system can be incorporated into existing workflows, enabling continuous monitoring of software quality throughout the development lifecycle. This supports agile and DevOps practices by providing timely feedback and improving collaboration among development teams. As a result, software products can be delivered with higher quality and reduced risk.

Finally, the project establishes a strong foundation for future advancements in software quality prediction systems. With further enhancements and integration of advanced technologies, the system can evolve into a comprehensive platform for intelligent software analytics.

Overall, the project demonstrates the practical applicability of machine learning in addressing real-world software engineering challenges and contributes to the development of smarter, more efficient, and reliable software systems. In summary, the proposed system demonstrates how modern machine learning techniques can be effectively utilized to improve software quality and reliability. By providing early risk prediction and actionable insights, the system supports developers in building more robust and maintainable software.

FUTURE ENHANCEMENT

The proposed system for software quality analytics and risk prediction can be further enhanced in several ways to improve its performance and usability. One of the major improvements is the integration of advanced machine learning algorithms such as XGBoost, Support Vector Machines, or deep learning models to achieve higher accuracy.

The system can be extended to use real-time data from software development tools like GitHub and CI/CD pipelines. This will enable continuous monitoring and prediction of software risks. Cloud integration can be implemented to make the system scalable and accessible from anywhere.

The current system uses a limited set of features, but additional software metrics can be included to improve prediction quality. Automated feature selection techniques can also be applied for better model performance. The system can be enhanced with a more interactive and dynamic user interface using modern frameworks like React or Angular.

Advanced data visualization techniques such as dashboards and analytics charts can be added for better insights. The model can be retrained periodically with updated datasets to maintain accuracy. Integration with DevOps tools can help in real-time decision-making during software development. The system can also include alert mechanisms to notify developers about high-risk modules.

Security features can be improved to protect user data and system integrity. Multi- user access with authentication can be implemented for better management. The application can be deployed as a Software-as-a-Service (SaaS) platform. Mobile application support can also be added for wider accessibility.

Natural Language Processing techniques can be integrated to analyze bug reports and logs. The system can be expanded to support multiple programming languages. Automated report generation can be included for documentation purposes. Performance optimization techniques can be applied for faster predictions. The system can also support integration with project management tools. Continuous feedback from users can be used to improve system features. Overall, future enhancements will make the system more intelligent, scalable, and suitable for real-world industrial applications.

Furthermore, the system can be enhanced by incorporating explainable artificial intelligence (XAI) techniques to improve transparency and trust in predictions. This will allow developers and project managers to understand the reasons behind risk classification and identify the most influential software metrics. Such interpretability will make the system more reliable for real-world industrial adoption. Additionally, hybrid models combining statistical methods and machine learning can be explored to further improve prediction accuracy and robustness. Another possible enhancement is the integration of automated data collection directly from software repositories and version control systems.

This will eliminate manual data entry and ensure that the system always works with up-to-date information. The inclusion of real-time streaming data processing frameworks can further improve system responsiveness. Moreover, incorporating anomaly detection techniques can help identify unusual patterns in software metrics that may indicate potential defects or vulnerabilities. The system can also be extended to include advanced performance optimization and distributed computing techniques. By leveraging technologies such as parallel processing and cloud-based infrastructure, the system can handle large datasets more efficiently. Integration with big data tools like Hadoop or Spark can

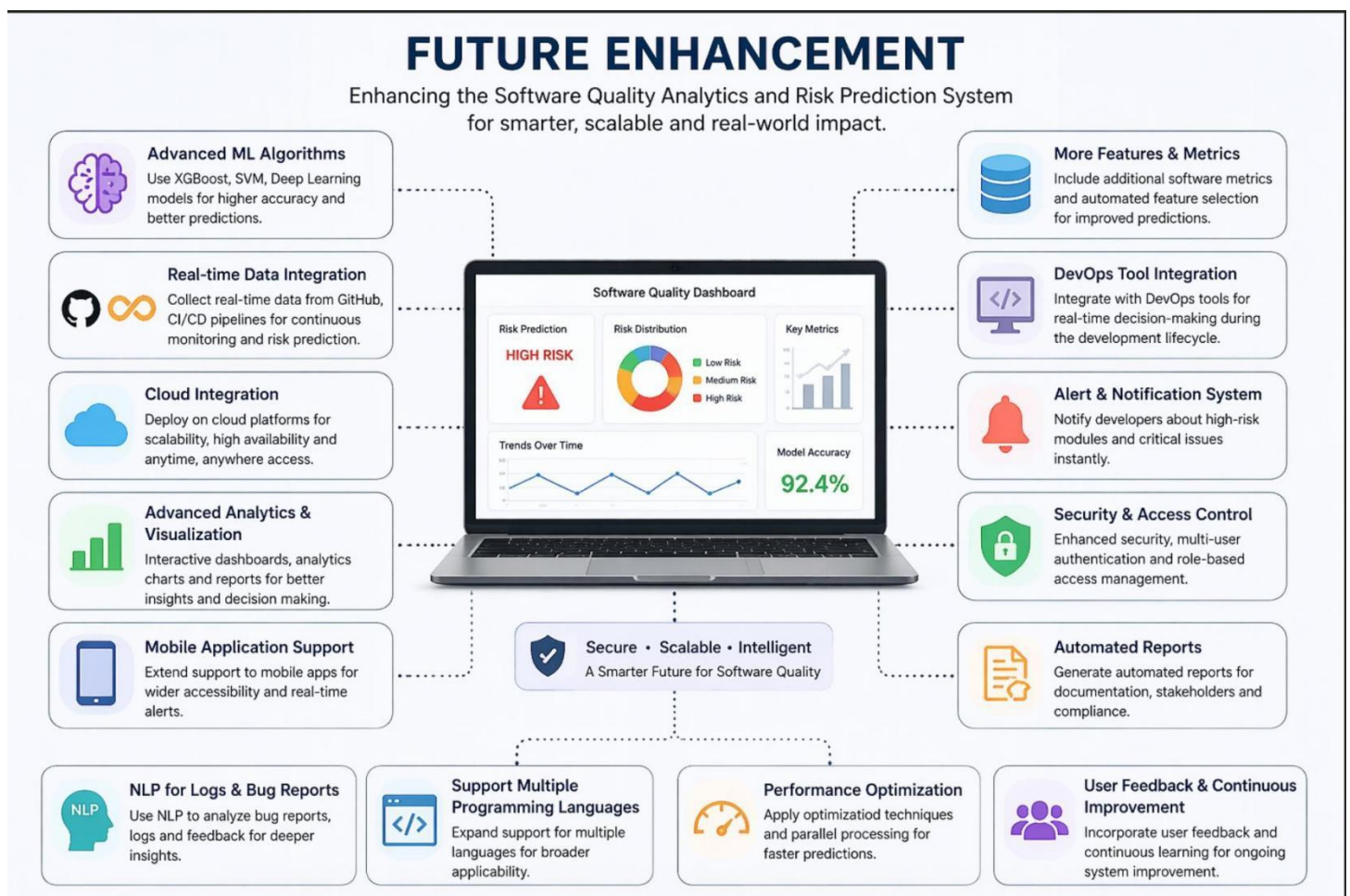
further enhance scalability and processing speed. This will make the system suitable for enterprise-level applications with massive volumes of software data.

In addition, the system can be improved by incorporating intelligent recommendation systems. These systems can suggest corrective actions, best coding practices, or refactoring techniques based on predicted risks. This will not only identify defects but also guide developers toward improving software quality. The inclusion of automated testing integration can further streamline the development process by linking prediction results with testing frameworks.

Finally, future enhancements can focus on improving user interaction and personalization. The system can provide customizable dashboards, role-based access control, and personalized insights for different stakeholders such as developers, testers, and project managers. Continuous learning mechanisms can also be implemented, where the system adapts based on user feedback and evolving project requirements. Overall, these enhancements will transform the system into a comprehensive, intelligent, and industry-ready software quality management platform. Moreover, the system can be enhanced by incorporating automated model selection and hyperparameter tuning techniques. This will allow the system to automatically choose the best-performing model and optimize its parameters without manual intervention. Such automation improves model efficiency and reduces the need for expert-level tuning. It also ensures that the system consistently delivers high prediction accuracy across different datasets. Another potential enhancement is the integration of federated learning techniques, where models can be trained across multiple distributed datasets without sharing sensitive data. This approach improves data privacy and security while still benefiting from diverse data sources. It is particularly useful for organizations that cannot share their internal data due to confidentiality concerns.

The system can also be extended to include predictive trend analysis, where future risk patterns are forecasted based on historical data. This will help organizations anticipate potential issues before they occur and take proactive measures. Trend analysis can provide valuable insights for long-term planning and risk management in software development.

Additionally, the implementation of a knowledge base system can further enhance usability. The system can store past predictions, solutions, and best practices, allowing users to refer to historical data for decision-making. This creates a learning environment where the system continuously improves over time and supports knowledge sharing among teams.



10.1 FUTURE ENHANCEMEN

REFERENCES

- [1] “XAI Driven Software Defect Prediction Using Adaptive Feature Engineering Coupled With Autoencoder and MLP,” *IEEE Access*, 2025.
- [2] M. Ali, T. Mazhar, Y. Arif, S. Al-Otaibi, Y. Y. Ghadi, T. Shahzad, M. A. Khan, and H. Hamam, “Software Defect Prediction Using an Intelligent Ensemble-Based Model,” *IEEE Access*, vol. 12, pp. 20376–20395, 2024.
- [3] T. Menzies, A. Butcher, A. Marcus, T. Zimmermann, and D. Cok, “Local versus global models for effort estimation and defect prediction,” *IEEE Transactions on Software Engineering*, 2011.
- [4] G. Boetticher, T. Menzies, and T. Ostrand, “PROMISE Repository of Software Engineering Databases,” West Virginia University, 2007.
- [5] NASA Metrics Data Program (MDP), “NASA Software Defect Datasets,” NASA, USA.
- [6] I. H. Witten, E. Frank, and M. A. Hall, *Data Mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann.
- [7] T. M. Mitchell, *Machine Learning*. McGraw-Hill Education.
- [8] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*. Elsevier.
- [9] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer.
- [10] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*. Springer.
- [11] L. Breiman, “Random Forests,” *Machine Learning*, 2001.
- [12] Scikit-learn, “Machine Learning in Python.” [Online]. Available: <https://scikit-learn.org>
- [13] Python Software Foundation, “Python Documentation.” [Online]. Available: <https://docs.python.org>
- [14] Flask, “Flask Documentation.” [Online]. Available: <https://flask.palletsprojects.com>
- [15] Kaggle, “Datasets.” [Online]. Available: <https://www.kaggle.com>
- [16] IEEE, “IEEE Xplore Digital Library.” [Online]. Available: <https://ieeexplore.ieee.org>

APPENDIX - I

ACCURACY LEVEL

```
===== MODEL PERFORMANCE =====  
Accuracy : 0.8461538461538461  
Precision: 1.0  
Recall   : 0.6666666666666666  
F1 Score : 0.8
```

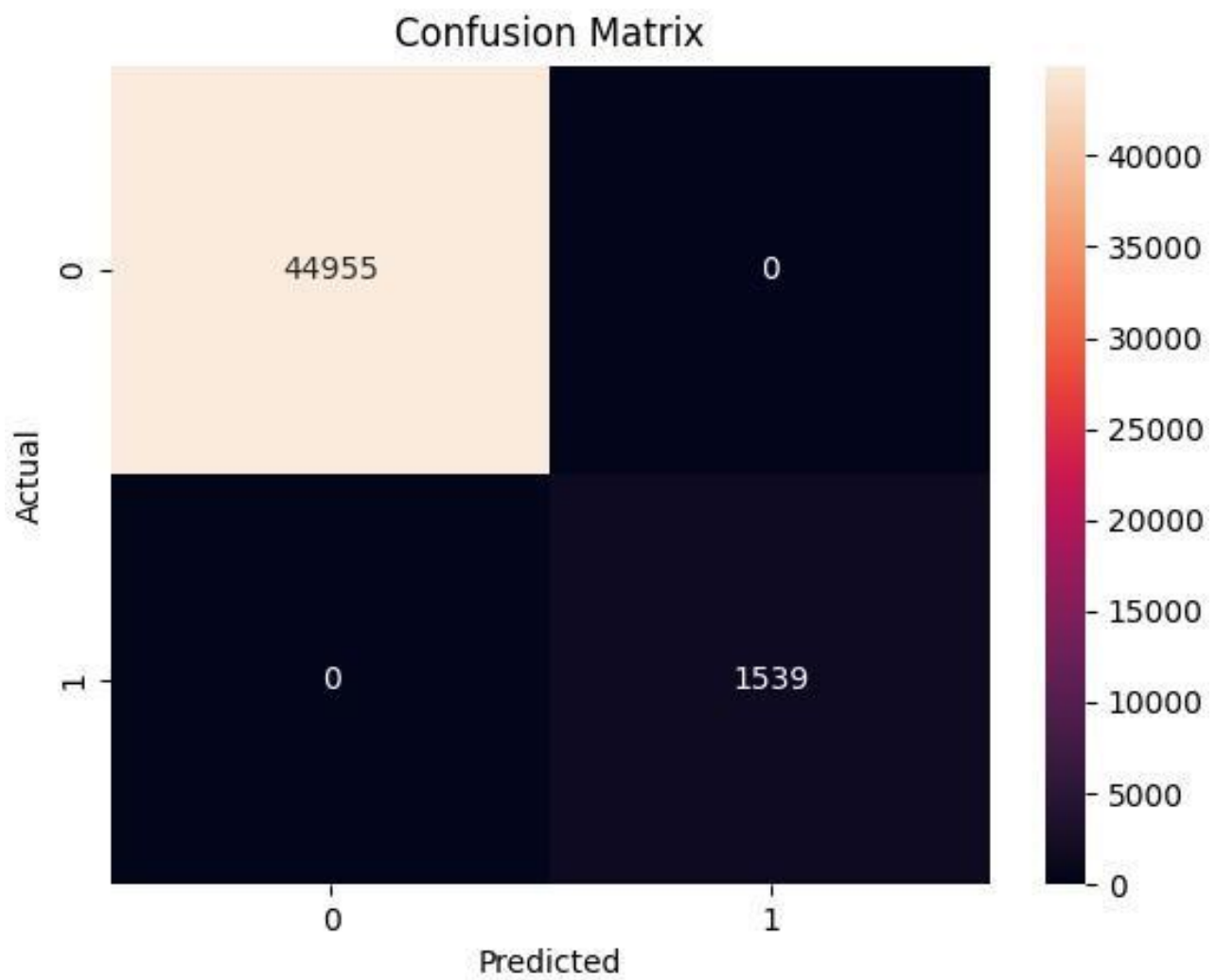
```
===== CLASSIFICATION REPORT =====
```

	precision	recall	f1-score	support
0	0.78	1.00	0.88	7
1	1.00	0.67	0.80	6
accuracy			0.85	13
macro avg	0.89	0.83	0.84	13
weighted avg	0.88	0.85	0.84	13

```
Model saved successfully!
```

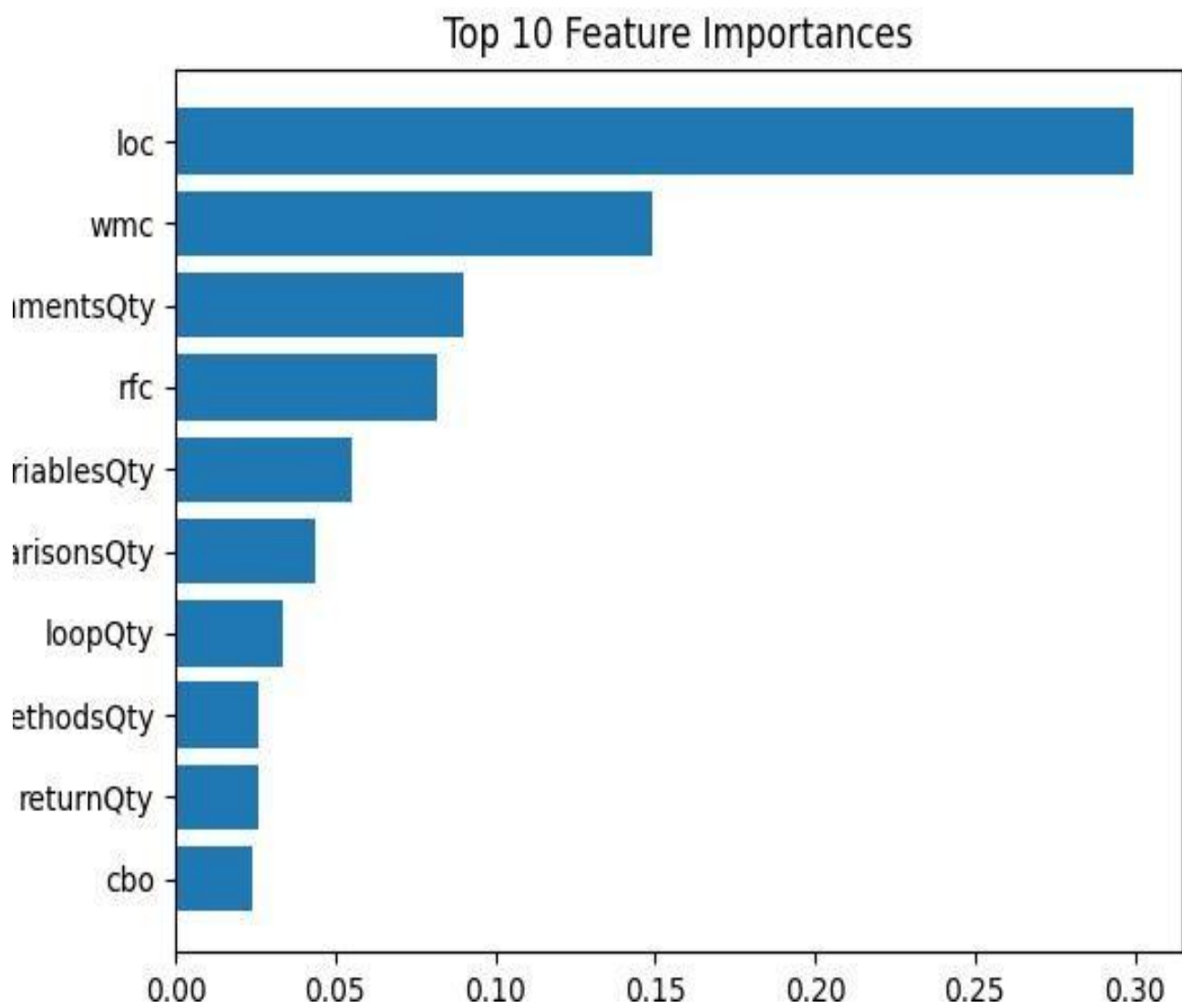
12.1.1 ACCURACY LEVEL

CONFUSION MATRIX



12.1.2 CONFUSION MATRIX

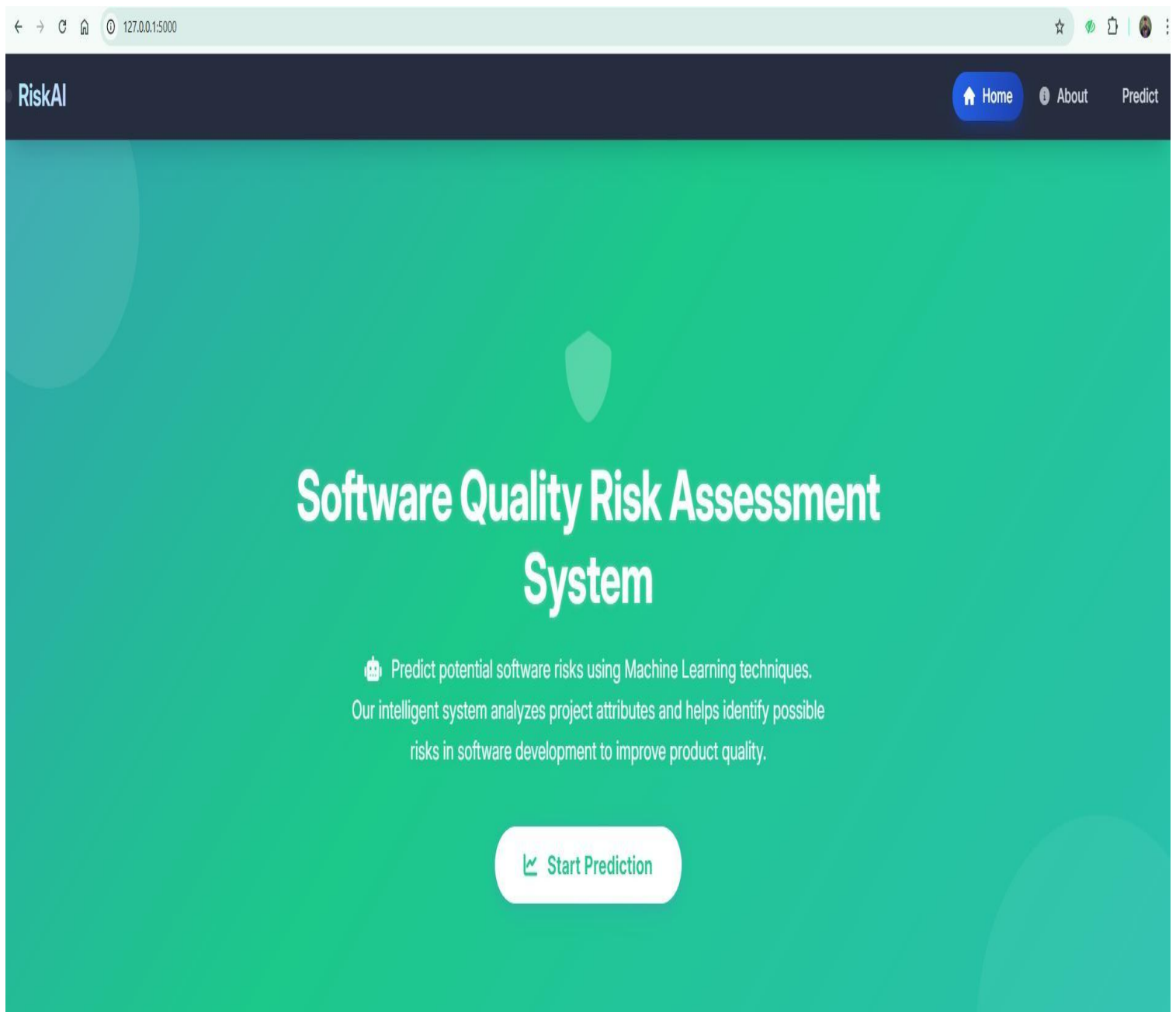
FEATURE IMPORTANCES



12.1.3 FEATURE IMPORTANCE

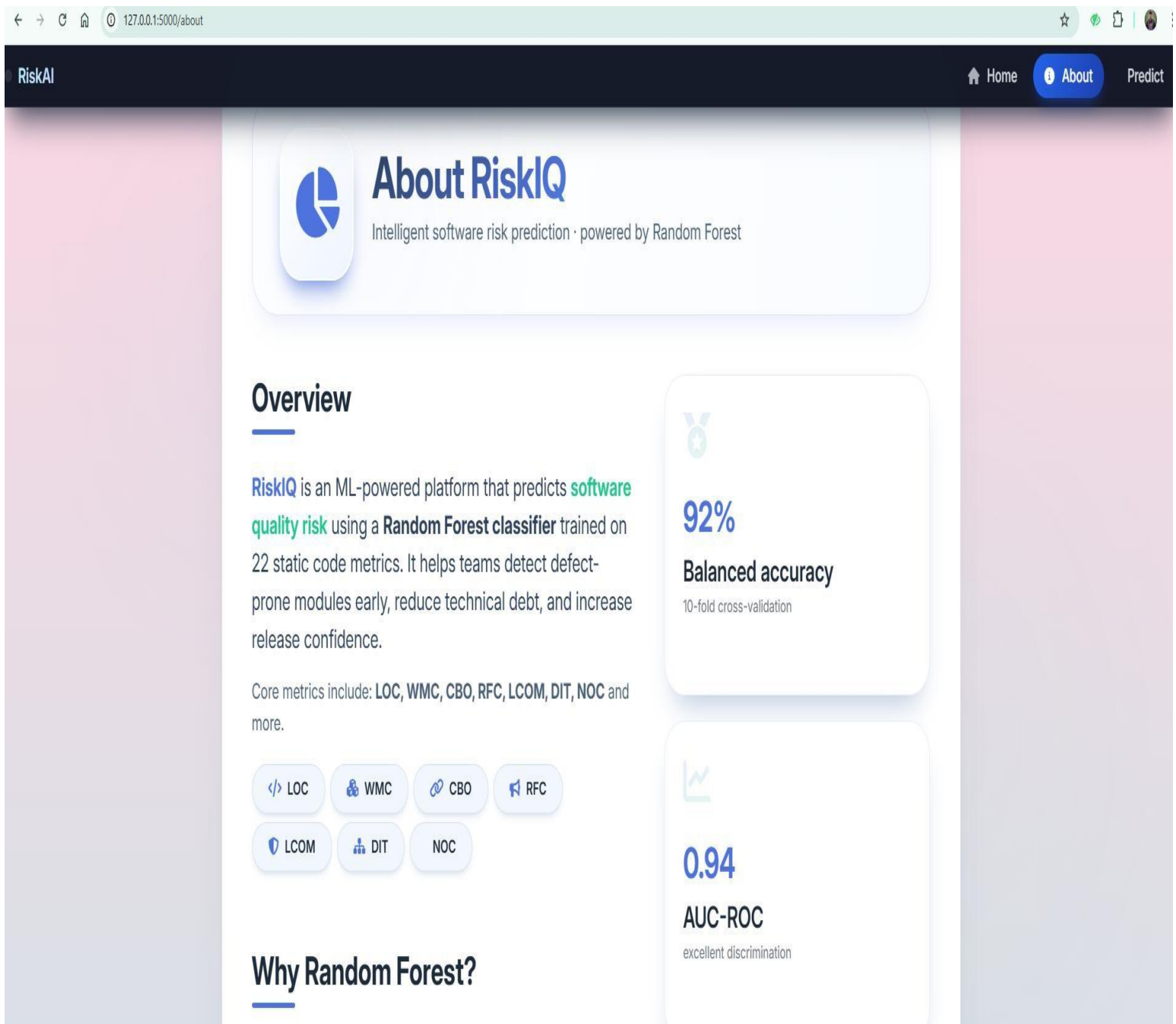
SCREENSHOTS

HOME PAGE



12.2.1 HOME PAGE

ABOUT PAGE



12.2.2 ABOUT PAGE

PREDICT PAGE

The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000/predict". The page has a dark blue header with the "RiskAI" logo on the left and navigation links for "Home", "About", and "Predict" on the right. The main content area features a white card titled "Predict Risk" with the subtitle "Enter software metrics below". Inside the card, there are twelve input fields arranged in a 4x3 grid, each with a software metric icon and name, an "Enter value" text box, and a unit selector dropdown. The metrics and their units are: Coupling Between Objects (CBO) - count, Fan-in - count, Fan-out - count, Weighted Methods (WMC) - count, Depth of Inheritance (DIT) - level, Number of Children (NOC) - count, Response for Class (RFC) - count, Lack of Cohesion (LCOM) - count, Lines of Code (LOC) - lines, Loop Quantity - count, Comparisons Quantity - count, and Max Nested Blocks - depth. At the bottom of the card is a large green button with a white flask icon and the text "Predict Risk".

Predict Risk
Enter software metrics below

Coupling Between Objects (CBO)	Fan-in
Enter value count	Enter value count
Fan-out	Weighted Methods (WMC)
Enter value count	Enter value count
Depth of Inheritance (DIT)	Number of Children (NOC)
Enter value level	Enter value count
Response for Class (RFC)	Lack of Cohesion (LCOM)
Enter value count	Enter value count
Lines of Code (LOC)	Loop Quantity
Enter value lines	Enter value count
Comparisons Quantity	Max Nested Blocks
Enter value count	Enter value depth

Predict Risk

12.2.3 PREDICT PAGE

VALUE ENTER

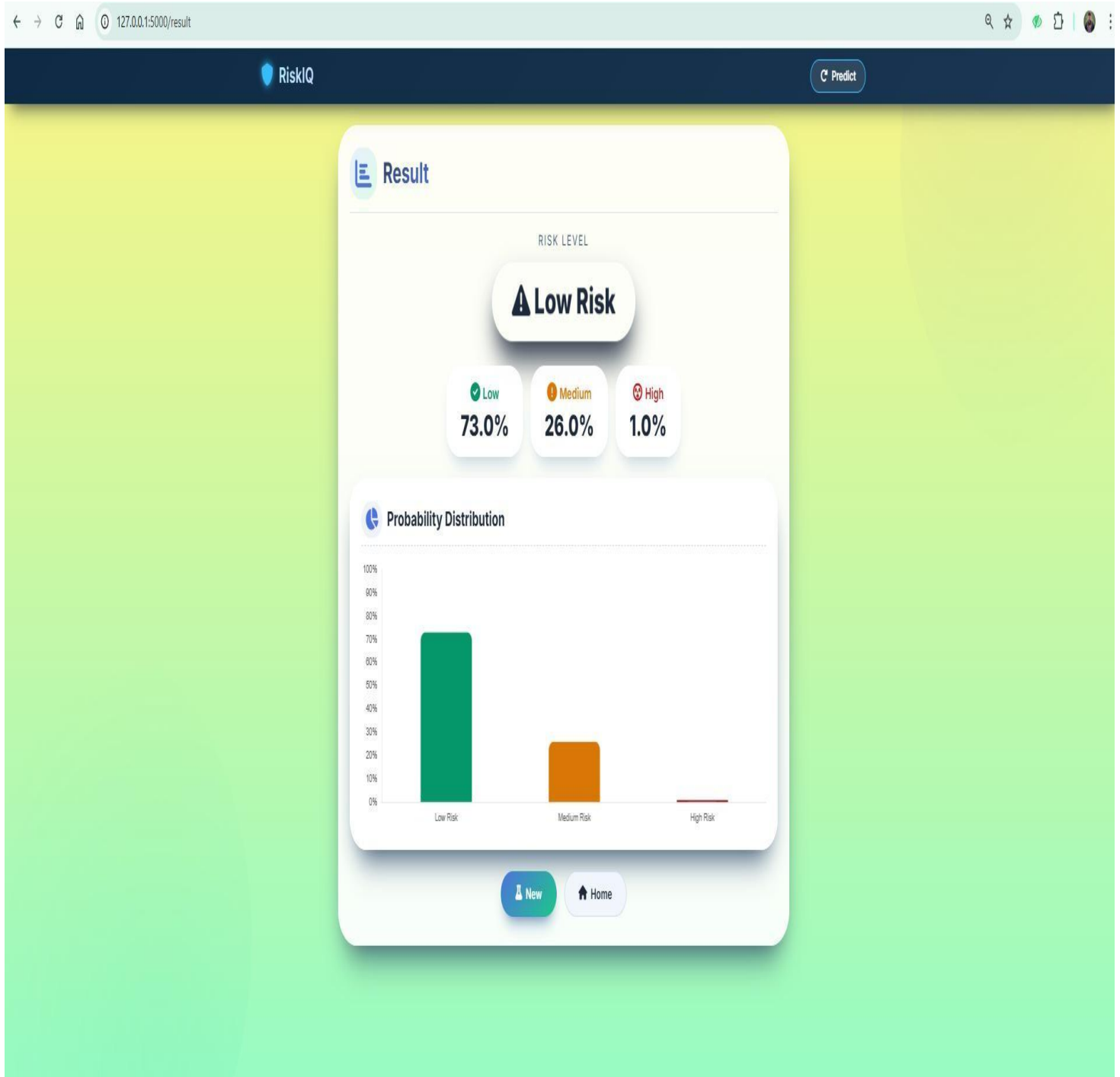
The screenshot shows a web browser window with the URL `127.0.0.1:5000/predict`. The page has a dark blue header with the text "RiskAI" on the left and navigation links "Home", "About", and "Predict" on the right. The main content area has a pink-to-blue gradient background. In the center is a white rounded rectangle titled "Predict Risk" with a subtitle "Enter software metrics below". Below the title are 12 input fields arranged in a 4x3 grid, each with a software metric icon and name. The inputs are: Coupling Between Objects (CBO) with value 453, Fan-in with 45, Fan-out with 21, Weighted Methods (WMC) with 12, Depth of Inheritance (DIT) with level 1, Number of Children (NOC) with 6545, Response for Class (RFC) with 221, Lack of Cohesion (LCOM) with 54, Lines of Code (LOC) with 24, Loop Quantity with 35, Comparisons Quantity with 321, and Max Nested Blocks with 53. The "Max Nested Blocks" input is highlighted with a green border. At the bottom of the white rectangle is a large blue button with a flask icon and the text "Predict Risk".

Metric	Value	Unit
Coupling Between Objects (CBO)	453	count
Fan-in	45	count
Fan-out	21	count
Weighted Methods (WMC)	12	count
Depth of Inheritance (DIT)	1	level
Number of Children (NOC)	6545	count
Response for Class (RFC)	221	count
Lack of Cohesion (LCOM)	54	count
Lines of Code (LOC)	24	lines
Loop Quantity	35	count
Comparisons Quantity	321	count
Max Nested Blocks	53	depth

Predict Risk

12.2.4 VALUE ENTER PAGE

RESULT PAGE



12.2.5 RESULTS PAGE

APPENDIX - II

```
from flask import Flask, render_template, request
import pandas as pd
import joblib
app = Flask(__name__)
# -----
# Load Model
# -----
model = joblib.load("software_risk_model.pkl")
# -----
# Load dataset to get columns
# -----
df = pd.read_csv("OnlyTrivial_dt.csv")
df = df.select_dtypes(include=['number'])
df['risk_level'] = 0
columns = df.drop('risk_level', axis=1).columns
risk_map = {
0: "Low Risk",
1: "Medium Risk",
2: "High Risk"
}
# -----

# Home Page
# -----
@app.route('/')
def home():
return render_template("home.html")
# -----

# About Page
# -----
@app.route('/about')
def about():
```

```

return render_template("about.html")

# -----

# Predict Page
# -----
@app.route('/predict')
def predict():
    return render_template("predict.html")
# -----

# Result Page
# -----
@app.route('/result', methods=['POST'])
def result():
    input_data = {col:0 for col in columns}
    input_data['cbo'] = float(request.form['cbo'])
    input_data['fanin'] = float(request.form['fanin'])
    input_data['fanout'] = float(request.form['fanout'])
    input_data['wmc'] = float(request.form['wmc'])
    input_data['dit'] = float(request.form['dit'])
    input_data['noc'] = float(request.form['noc'])
    input_data['rfc'] = float(request.form['rfc'])
    input_data['lcom'] = float(request.form['lcom'])
    input_data['loc'] = float(request.form['loc'])
    input_data['loopQty'] = float(request.form['loopQty'])
    input_data['comparisonsQty'] = float(request.form['comparisonsQty'])
    input_data['maxNestedBlocksQty'] = float(request.form['maxNestedBlocksQty'])
    data = pd.DataFrame([input_data])
    data = data[columns]
    prediction = model.predict(data)[0]
    prob = model.predict_proba(data)[0]
    labels = ["Low Risk", "Medium Risk", "High Risk"]
    return render_template(
        "result.html",
        prediction=risk_map[prediction],

```

```

labels=labels,
prob=prob.tolist()
)
# -----

# Run App
# -----
if __name__ == "__main__":
app.run(debug=True)
import pandas as pd
import numpy as np
import joblib
import matplotlib.pyplot as plt
# -----

# Load Model
# -----
model = joblib.load("software_risk_model.pkl")
# -----

# Load Dataset to get columns
# -----
df = pd.read_csv("OnlyTrivial_dt.csv")
df = df.select_dtypes(include=[np.number])

df['risk_level'] = 0
columns = df.drop('risk_level', axis=1).columns
# -----

# Create input dictionary
# -----
input_data = {col:0 for col in columns}
print("\nEnter Important Software Metrics\n")
input_data['cbo'] = float(input("Enter cbo: "))
input_data['fanin'] = float(input("Enter fanin: "))
input_data['fanout'] = float(input("Enter fanout: "))
input_data['wmc'] = float(input("Enter wmc: "))
input_data['dit'] = float(input("Enter dit: "))

```

```

input_data['noc'] = float(input("Enter noc: "))
input_data['rfc'] = float(input("Enter rfc: "))
input_data['lcom'] = float(input("Enter lcom: "))
input_data['loc'] = float(input("Enter loc: "))
input_data['loopQty'] = float(input("Enter loopQty: "))
input_data['comparisonsQty'] = float(input("Enter comparisonsQty: "))
input_data['maxNestedBlocksQty'] = float(input("Enter maxNestedBlocksQty: "))
# -----
# Convert to DataFrame
# -----
data = pd.DataFrame([input_data])
data = data[columns]
# -----
# Prediction
# -----
prediction = model.predict(data)[0]
prob = model.predict_proba(data)[0]
risk_map = {0:"Low Risk",1:"Medium Risk",2:"High Risk"}
print("\nPredicted Software Risk:", risk_map.get(prediction))
-----
# Graph Labels
# -----
classes = model.classes_
labels = [risk_map[i] for i in classes]
# -----
# Crypto Style Prediction Graph
# -----
plt.style.use("dark_background")
plt.figure(figsize=(8,5))
plt.plot(labels, prob, marker='o', linewidth=3)
plt.fill_between(labels, prob, alpha=0.3)
plt.title("Software Risk Prediction Probability", fontsize=14)
plt.xlabel("Risk Level")
plt.ylabel("Probability")

```

```

plt.grid(True)
plt.show()
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, recall_score, accuracy_score,
classification_report
# -----
# 1. Load Dataset
# -----
df=pd.read_csv("OnlyTrivial_dt.csv")
# -----
# 2. Drop Non-Numeric Columns
# -----
df=df.select_dtypes(include=[np.number])
# -----
# 3. Create Risk Label
# -----
df['risk_level']=0
# Medium Risk
df.loc[(df['loc'] > 200) & (df['loc'] <= 500), 'risk_level'] = 1
# High Risk
df.loc[(df['loc'] > 500), 'risk_level'] = 2
-----
# 4. Check Risk Distribution
# -----
print("\nRisk Level Distribution:\n")
print(df['risk_level'].value_counts())
# -----

```

```

# 5. Define X and Y
# -----
X = df.drop('risk_level', axis=1)
y = df['risk_level']
# -----

# 6. Train Test Split
# -----
X_train, X_test, y_train, y_test =
train_test_split( X, y, test_size=0.2,
random_state=42)
# -----

# 7. Random Forest Model
# -----
model =
RandomForestClassifier( n_estim
ators=100, random_state=42
)
# -----

# 8. Predictions
# -----
y_pred = model.predict(X_test)
# -----

# 9. Evaluation
# -----
accuracy = accuracy_score(y_test, y_pred)
recall = recall_score(y_test, y_pred, average='macro')
print("\nAccuracy:", accuracy)
print("Recall:", recall)

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
# -----

# 10. Confusion Matrix
# -----

```

```

cm = confusion_matrix(y_test, y_pred)
plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# -----
# 11. Feature Importance Graph
# -----

importances = model.feature_importances_
indices = np.argsort(importances)[-10:]
plt.figure()
plt.barh(range(len(indices)), importances[indices])
plt.yticks(range(len(indices)), X.columns[indices])
plt.title("Top 10 Important Features")
plt.show()

# -----
# 12. Save Model
# -----

joblib.dump(model, "software_risk_model.pkl")
print("\nModel Saved Successfully!")
-7">

```

<h2 class="section-title-modern">Overview</h2>

<p class="fs-4" style="color: #2c3f5c; line-height: 1.5;">

RiskIQ is an ML-powered platform that predicts

software quality risk using a

Random Forest classifier trained on 22 static code metrics. It helps teams detect

defect-prone modules early, reduce technical debt, and increase release confidence.

</p>

<p class="fs-5" style="color: #4a5f7a;">

Core metrics include: LOC, WMC, CBO, RFC, LCOM, DIT, NOC and more.

</p>

<!-- metric chips -->

<div class="d-flex flex-wrap gap-2 my-4">

<i class="fas fa-code"></i> LOC

<i class="fas fa-cubes"></i> WMC

<i class="fas fa-link"></i> CBO

<i class="fas fa-bullhorn"></i> RFC

<i class="fas fa-shield"></i> LCOM

<i class="fas fa-sitemap"></i> DIT

<i class="fas fa-children"></i> NOC

</div>

<!-- why random forest -->

<h2 class="section-title-modern mt-5">Why Random Forest?</h2>

<ul class="feature-list-modern">

<i class="fas fa-check-circle"></i> Handles non-linear relationships among 20+ metrics

<i class="fas fa-check-circle"></i> Built-in feature importance & SHAP explainability

<i class="fas fa-check-circle"></i> Robust to outliers and imbalanced datasets

<i class="fas fa-check-circle"></i> Outputs probability scores (0–100% risk) for nuanced decisions

</div>

<div class="col-lg-5 d-flex flex-column gap-4">

<div class="insight-card-modern">

<i class="fas fa-medal"></i>

<div class="fs-1 fw-bold" style="color:#4e73df;">92%</div>

<div class="fs-3 fw-semibold">Balanced accuracy</div>

10-fold cross-validation

</div>

<div class="insight-card-modern">

```

<i class="fas fa-chart-line"></i>
<div class="fs-1 fw-bold" style="color:#4e73df;">0.94</div>
<div class="fs-3 fw-semibold">AUC-ROC</div>
<span class="text-secondary">excellent discrimination</span>
</div>
<div class="insight-card-modern">
<i class="fas fa-tree"></i>
<div class="fs-1 fw-bold" style="color:#4e73df;">200</div>
<div class="fs-3 fw-semibold">estimators</div>
<span class="text-secondary">ensemble depth = 12</span>
</div>
</div>
</div>
<div class="divider-modern"></div>
<!-- ===== DATASET & INTERPRETABILITY ===== -->
<div class="row g-5 align-items-stretch">
<div class="col-md-6">
<h2 class="section-title-modern">Training dataset</h2>
<div class="dataset-modern">
<i class="fas fa-database fa-4x mb-4" style="color:#4e73df;"></i>
<h3 class="fw-bold mb-3" style="color:#1e2b3c;">NASA MDP + PROMISE</h3>
<p class="fs-5 mb-4" style="color:#4a5f7a;">Augmented with 5,000+ modern open-source
Java projects from GitHub.</p>
<div class="d-flex gap-4 flex-wrap fs-5">
<span><i class="fas fa-layer-group me-2" style="color:#1cc88a;"></i> 18k+
instances</span>
<span><i class="fas fa-tag me-2" style="color:#1cc88a;"></i> 22 metrics</span>
<span><i class="fas fa-balance-scale me-2" style="color:#1cc88a;"></i> SMOTE-
balanced</span>
</div>
<hr class="opacity-25">
<p class="mb-0"><i class="fas fa-flask me-2" style="color:#4e73df;"></i>Preprocessing:
normalization, correlation filtering, feature selection.</p>

```

```

</div>
</div>
<div class="col-md-6">
<h2 class="section-title-modern">Interpretability</h2>
<div class="dataset-modern">
<i class="fas fa-lightbulb fa-3x mb-3" style="color: #f6b83e;"></i>
<p class="mt-2 fs-4 fw-semibold">Feature importance & SHAP</p>
<!-- importance bars -->
<div class="mt-4">
<div class="d-flex align-items-center mb-3"><span style="width:100px;">CBO:</span>
<div class="ms-3 bar-modern" style="width:82%;"></div></div>
<div class="d-flex align-items-center mb-3"><span style="width:100px;">RFC:</span> <div
class="ms-3 bar-modern" style="width:74%;"></div></div>
<div class="d-flex align-items-center mb-3"><span style="width:100px;">WMC:</span>
<div class="ms-3 bar-modern" style="width:88%;"></div></div>
<div class="d-flex align-items-center"><span style="width:100px;">LOC:</span> <div
class="ms-3 bar-modern" style="width:69%;"></div></div>
</div>
<p class="text-secondary mt-4"><i class="fas fa-chart-simple me-2"></i> mean SHAP
values (top features)</p>
</div>
</div>
</div>
<div class="divider-modern"></div>
<!-- ===== TECH STACK + HOME BUTTON ===== -->
<div class="row align-items-center">
<div class="col-lg-8">
<h2 class="section-title-modern">Tech stack</h2>
<div class="d-flex flex-wrap gap-3">
<span class="tech-badge-modern"><i class="fab fa-python"></i> Python / scikit-
learn</span>
<span class="tech-badge-modern"><i class="fas fa-brain"></i> Random Forest</span>
<span class="tech-badge-modern"><i class="fas fa-chart-bar"></i> Pandas / NumPy</span>
<span class="tech-badge-modern"><i class="fas fa-flask"></i> SHAP / Lime</span>

```

```

<span class="tech-badge-modern"><i class="fab fa-html5"></i> Flask / Bootstrap</span>
</div>
</div>
<div class="col-lg-4 text-lg-end mt-4 mt-lg-0">
<a href="/" class="home-button-modern">

<i class="fas fa-arrow-left"></i>Back to Home</a> </div>

</div>
<!-- footer note (subtle) -->
<div class="text-center text-muted mt-5 small" style="opacity: 0.6;">
<i class="fas fa-shield-alt me-2 floating-icon"></i> RiskIQ · version
4.0 · continuously improving </div>

</main>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"
></script>
</body>
</html>

```

(12) PATENT APPLICATION PUBLICATION

(21) Application No.202641027375 A

(19) INDIA

(22) Date of filing of Application :09/03/2026

(43) Publication Date : 20/03/2026

(54) Title of the invention : Software Quality Analysis and Risk Assessment Using Cloud-Based Machine Learning

(51) International classification	:G06F 11/36, G06N 20/00, G06F 9/44, G06Q 10/06, G06F 21/57	(71) Name of Applicant : 1)DIVA SENTHILNATHAN Address of Applicant :V.S.B. COLLEGE OF ENGINEERING TECHNICAL CAMPUS, NH-209, Coimbatore-Pollachi main road, Ealur Pirivu, Solavampalayam (po), Tamil Nadu India
(31) Priority Document No	:NA	(72) Name of Inventor :
(32) Priority Date	:NA	1)DR.R.JAMUNA
(33) Name of priority country	:NA	2)DR.P.RAJU
(86) International Application No	:	3)DR T KALAIKUMARAN
Filing Date	:01/01/1900	4)MRS V RADHA
(87) International Publication No	: NA	5)R.R.PRASANNARAM
(61) Patent of Addition to Application Number	:NA	
Filing Date	:NA	
(62) Divisional to Application Number	:NA	
Filing Date	:NA	

(57) Abstract :

Software quality has a critical role to play in the success of present-day applications. However, many organizations still rely on manual code reviews, static testing tools, and quality-checking measures after software development to detect defects. Their results are often obtained at later stages of the development process, which increases the cost of the project and delays the delivery date. This project suggests a cloud-based machine learning system which can be used for assessing software quality and determining the risks involved in the software development process. Instead of waiting for failures to manifest, the system predicts defect modules, security defects, and performance risks in the early stage. Such a risk ranking system is aimed at being based on the analysis of the source code metrics, historical defect records, test coverage reports, and development activity logs. The solution is deployed on a cloud platform, which enables scalable processing, data storage in one place, and easy integration with development pipelines. A web dashboard delivers clear visual insights that give developers, testers, and project managers access to faster and more informed decision-making. This transition from reactive test cases to risk prediction results in fewer failures after release, better maintainability, and enhanced overall reliability.

No. of Pages : 8 No. of Claims : 10